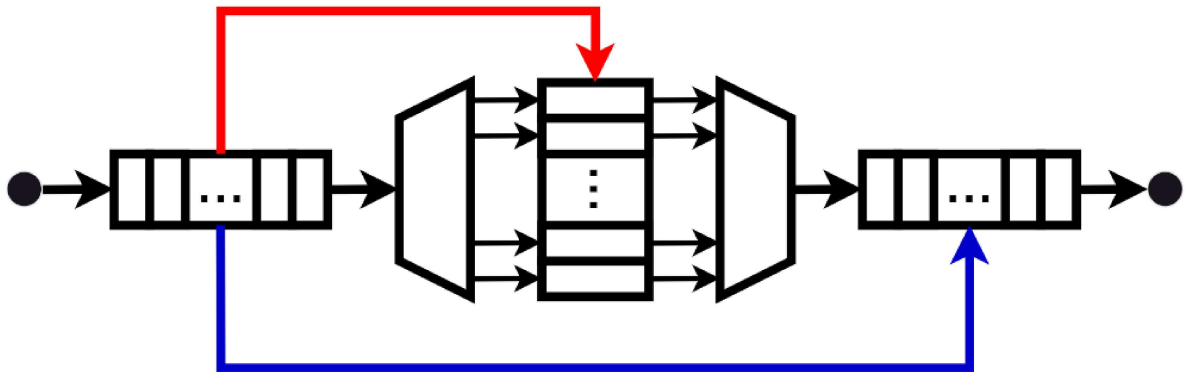


DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Spring Semester 2024

Deterministic Throughput for GRAND

Bachelor Thesis

Filippo Christen
fchrist@student.ethz.ch

June 2024

Supervisors: Darja Nonaca, dnonaca@iis.ee.ethz.ch

Professor: Prof. Dr. Christoph Studer, studer@ethz.ch

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Darja Nonaca, for her kindness and unwavering support throughout this Bachelor's Thesis. Her guidance has been crucial to the success of this work.

I am also deeply grateful to Prof. Dr. Christoph Studer for his time and insightful feedback. His expertise and the opportunity he provided for me to undertake this project have been greatly appreciated.

Abstract

A FIFO-Scheduling approach in mitigating the nondeterministic running time of GRAND decoding schemes is simulated and explored. Ultra-Reliable Low-Latency Communication (URLLC) based contemporary applications such as Internet of Things and Virtual Reality call for high-rate, short error correction codes, which raises the possibility of practical universal decoders being revisited. Proposed code agnostic decoding algorithms such as Guessing Random Additive Decoding (GRAND) provide near-ML decoding performance, while yielding practically realizable hardware implementations. However, their iterative execution paired with the inherent randomness of noise, causes the latency of GRAND decoders to be nondeterministic, which proves problematic in real-world applications. FIFO-Scheduling is investigated as a low-overhead solution to efficiently enforce deterministic throughput for GRAND algorithms, while avoiding the introduction of a hard-coded iteration ceiling.

Contents

1	Introduction	1
2	Theory	3
2.1	GRAND	3
2.2	ORBGRAND	4
2.2.1	Motivation	4
2.2.2	Principles	5
2.2.3	Reliability weight	6
2.2.4	Noise effect sequences generation	8
2.2.5	Landslide algorithm	12
2.3	FIFO-Scheduling	14
2.3.1	Motivation	14
2.3.2	Structure	15
2.3.3	Early Termination	15
3	ORBGRAND Simulation	17
3.1	Overview	17
3.2	Reliability curve approximation	18
3.3	Putative noise generation	19
3.4	Memoization	20
4	FIFO Scheduling Simulation	21
4.1	Overview	21
4.2	Throughput simulation	22
4.2.1	FIFO	23
4.2.2	Decoders	24
4.2.3	Re-Ordering buffer	24
4.2.4	Booking mechanism	25
4.2.5	Early termination mechanism	25
4.3	Performance indicators	27

Contents

5	Results	29
5.1	ORBGRAND Results	29
5.2	FIFO-Scheduling results	31
5.2.1	Overview	31
5.2.2	Latency	34
5.2.3	Area	34
5.2.4	Storage	35
5.2.5	Area-Time product	36
5.2.6	Early termination mechanism	37
6	Conclusion and Future Work	41
6.1	Summary	41
6.2	Conclusions	42
6.2.1	ORBGRAND	42
6.2.2	FIFO-Scheduling	42
6.3	Future Work	43

List of Figures

2.1	Reliability curve approximations of a data vector of length $n = 256$ for different E_b/N_0 values, scaled to allow a better illustration.	7
2.2	Ferris diagram of Landslide algorithm. The upward arrow indicates the corresponding part is to be increased by 1 in the next step.	14
2.3	FIFO-Scheduling Structure, red arrow: Early termination mechanism, blue arrow: Booking mechanism	16
2.4	Early termination flow chart	16
3.1	Geometric and linear anchoring comparison. Scaled to improve illustration.	18
4.1	Re-ordering buffer in action	24
4.2	Booking mechanism.	26
5.1	Comparison of decoding accuracy and mean required iterations across different segment numbers and anchoring methods.	30
5.2	Required iterations CDF at different E_b/N_0 values. red dashed line at the mean. Decoder is geometrically anchored 4 segment ORBGRAND, for 10^4 decodings at iteration ceiling 10^9	31
5.4	Contruction of $BLER = 10^{-3}$ intersect $\frac{E_b}{N_0}$ data. Reference is given in blue.	38
5.5	Pareto Maximum Latency plot, (F,D,R)	39
5.6	Pareto Area plot, (F,D,R)	39
5.7	Pareto Storage plot, (F,D,R)	39
5.8	Pareto Area-Time product plot, (F,D,R)	40
5.9	FIFO occupancy, over $4 \cdot 10^5$ cycles. Complete simulation involved 10^4 decodings.	40

Introduction

Shannon’s pioneering work [1] established that the maximum code rate a channel can support is achieved with long codes. However, since 1978, it has been known that optimal Maximum Likelihood (ML) decoding of linear codes is an NP-complete problem [2]. These findings have led to an engineering paradigm that involves co-designing specific classes of linear codebooks alongside code-specific decoders. The resulting architectures leverage the code structure to facilitate computationally efficient approximate-ML decoding for long, high-redundancy codes [3].

Recent applications such as augmented and virtual reality, vehicle-to-vehicle communications, machine-type communications and Internet of Things [4, 5, 6, 7, 8], have fueled the demand for Ultra-Reliable Low-Latency Communication (URLLC). The requirement for shorter and thus less complex codes, imposed by these novel use cases, allows for the possibility of high-accuracy near-optimal universal decoders to be revisited. The development of practical universal decoders would significantly expand the range of potential code-books that can be decoded with a single algorithmic instantiation, thereby minimizing hardware footprint, ensuring devices are future-proofed against new codes and providing the flexibility for each application to choose the most suitable code-book [9].

Paramount to the realization of such universal decoders is the development of algorithms which are inherently suitable for hardware implementation. One such approach is Guessing Random Additive Noise Decoding (GRAND), first introduced in [10], which has been shown to provide ML decodings for any moderate redundancy block-code construction [11]. GRAND achieves this by iteratively subtracting putative noise effect sequences from a given received demodulated data vector $\mathbf{y}$, and consequently checking if the resulting bitstring is part of the given codebook. The first such codeword is then picked as the decoding. The algorithm’s simplicity and parallelizability, make GRAND suitable for hardware implementation, which resulted in VLSI implementation proposals [12, 13] and realization [14].

1 Introduction

A soft information incorporating version of GRAND, which lends itself to hardware implementation, while achieving approximate soft-ML decoding performance is Ordered Reliability Bits GRAND (ORBGRAND) [15, 9], which relies on a piecewise linear approximation of the real-valued soft information associated to each demodulated bit. Several ORBGRAND VLSI realizations have been proposed [16, 17, 18, 19], demonstrating that the algorithm lends itself to hardware implementation. These properties make ORBGRAND a desirable candidate for a practical, high-accuracy near-optimal universal decoder, which utilizes soft decoding information.

ORBGRAND’s iterative execution based on repeated noise-effect sequence trials, paired with the intrinsic randomness of channel noise effects on any given transmitted data vector, causes the required decoding iterations for each data vector to be nondeterministic. This property hinders a straight-forward integration of ORBGRAND decoders into real-world systems, where deterministic throughput is strictly required. To mitigate this limitation, in this work, we explore FIFO-Scheduling as introduced in [20], as a possible approach to reach deterministic throughput for ORBGRAND decoders, without introducing a hard-coded upper limit to the number of iterations a given decoder is allowed to perform. This approach relies on the cumulative benefits of input buffering, parallel decoding and output re-ordering to insulate the surrounding system from the irregular latency intrinsic to GRAND algorithms, while allowing for a flexible number of decoding iterations for each data vector.

Aim of this work is to explore and compare various FIFO-Scheduling architectures for ORBGRAND decoders, to demonstrate the effectiveness of this approach, while underlining some of its fundamental tradeoffs. Through simulation it is shown that even small versions of FIFO-Scheduling enabled ORBGRAND, result in considerable performance improvements over a simple hard-coded iteration ceiling.

Theory

2.1 GRAND

Any real-world data transmission is inherently noisy, meaning that transmitted data on its journey from transmitter to receiver could potentially be altered. To mitigate data corruption, error correction codes are employed, which encode any data vector $\mathbf{b}$ of length k , by introducing redundancy bits at the transmitter. The resulting codeword $\mathbf{x}$ will be such that its length n is larger than k , allowing the decoder within the receiver to detect whether a given detected data vector is corrupted, or even recover the originally transmitted codeword $\mathbf{x}$. Denoting the received hard decision demodulated data vector as $\mathbf{y}$, it is possible to express the noise effect by the channel on $\mathbf{x}$ as $\mathbf{y} = \mathbf{x} \oplus \mathbf{z} \Leftrightarrow \mathbf{z} = \mathbf{y} \oplus \mathbf{x}$. Where $\oplus$ denotes addition in the second order Galois field $\mathbb{F}_2$ and $\mathbf{z}$ denotes the noise effect sequence introduced by the channel. To reconstruct the originally transmitted codeword $\mathbf{x}$ out of the received demodulated data vector $\mathbf{y}$, the decoder aims at finding the maximum likelihood, or ML-decoding, given by

$$\mathbf{x}_{ML} = \arg \max_{\mathbf{x} \in C} P(\mathbf{y}|\mathbf{x}) = \arg \max_{\mathbf{x} \in C} P(\mathbf{z} = \mathbf{y} \oplus \mathbf{x}) \quad (2.1)$$

Finding such an optimal $\mathbf{x}_{ML}$ through a brute-force approach is practically impossible even for relatively short codes, as all $|C| = 2^k$ possible codewords $\mathbf{x} \in C$ would have to be compared.

Guessing random additive noise decoding or GRAND, is a code agnostic decoder which operates by iteratively adding putative noise effect sequences $\mathbf{z}$ to received data vectors $\mathbf{y}$ and subsequently checking if the resulting datavector $\hat{\mathbf{x}} = \mathbf{y} \oplus \mathbf{z}$ is a valid codeword such that $\hat{\mathbf{x}} \in C$, by means of a code specific, code-book membership test function. The pseudo-code of a general GRAND algorithm is given in Algorithm 1. The putative

2 Theory

noise effect sequences are tested in decreasing order of likelihood such that $P(\mathbf{Z} = \mathbf{z}^\alpha) \geq P(\mathbf{Z} = \mathbf{z}^\gamma) \forall \alpha < \gamma$, where α and γ are the ordering indexes. This implies that the first found codeword is an ML-decoding, even in channels with memory and in the absence of interleaving [21, 22].

For a code where bitstrings of length k are encoded into codewords of length n , all GRAND algorithms identify a decoding after approximatively geometrically distributed number of code-book queries with mean 2^{n-k} [9], theorem 2. This implies that the complexity of GRAND algorithms is dictated not by the codeword length of a given error correction code, but rather by its associated number of redundancy bits $n - k$, making them suitable for decoding any moderate redundancy codeword.

Algorithm 1: Guessing Random Additive Noise Decoding (GRAND)

Input: Code-book membership function $C_{\text{memb}} : \{0, 1\}^n \mapsto \{0, 1\}$; demodulated bits $\mathbf{y}$; optional information Φ .

Output: Decoding $\mathbf{x}_{ML}$

```

1  $d \leftarrow 0$ ;
2 while  $d = 0$  do
3    $\mathbf{z} \leftarrow$  next most likely binary noise effect sequence;
4   if  $C_{\text{memb}}(\mathbf{y} \oplus \mathbf{z}) = 1$  then
5      $\mathbf{x}_{ML} \leftarrow \mathbf{y} \oplus \mathbf{z}$ ;
6      $d \leftarrow 1$ ;
7   end
8 end
9 return  $\mathbf{x}_{ML}$ 

```

2.2 ORBGRAND

2.2.1 Motivation

While GRAND provides hard ML-decoding performance, readily available soft information remains unused, which can be wasteful as incorporating soft information into the decoding process is known to provide performance improvements [23, 24, 25]. For GRAND algorithms this effectively means utilizing the soft channel information to adjust the putative noise effect sequence trial ordering in an appropriate way. Symbol Reliability GRAND (SRGRAND) [26, 27], is one such approach, which adds one single bit for each of the received n data vector bits, which mark if the associated demodulated bit is to be considered reliable or not. SRGRAND retains the desirable simplicity and parallelizability offered by the original algorithm, while improving decoding performance by 0.5 - 0.75 dB over hard detection GRAND [27]. A further approach is Soft GRAND (SGRAND) [28], which aims at utilizing the full available soft channel information, by

2 Theory

associating real-valued soft information for each demodulated bit, generating a dedicated noise effect sequence ordering for each received data vector $\mathbf{y}$. Semi-parallelizable implementations are possible in software, and being a true soft-ML decoder, SGRAND provides a benchmark for optimal decoding accuracy performance. However, SGRAND's execution is algorithmically involved and does not lend itself to hardware implementation [28].

A GRAND version which utilizes soft information, while retaining hardware implementability, is Ordered Reliability GRAND (ORBGRAND). First proposed in [15], it bridges the gap between SRGRAND and SGRAND, providing approximate soft-ML decoding performance and hardware implementability at the same time.

2.2.2 Principles

Data transmission of a codeword $\mathbf{x}$ over a communication system, involves modulation of $\mathbf{x}$ into a symbol vector, which will be denoted as $\tilde{\mathbf{x}}$. The noisy nature of the channel modeled as Additive White Gaussian Noise (AWGN), implies that the datavector arriving at the receiver will amount to $\tilde{\mathbf{y}} = \tilde{\mathbf{x}} + \mathbf{n}$, where $\mathbf{n}$ represents the real valued noise on each transmitted symbol. The incoming $\tilde{\mathbf{y}}$ is then demodulated into $\mathbf{y}$ through hard decision demodulation and can be represented as $\mathbf{y} = \mathbf{x} \oplus \mathbf{z} \Leftrightarrow \mathbf{z} = \mathbf{y} \oplus \mathbf{x}$, where $\mathbf{z}$ represents the effect the channel noise had on the transmission. All GRAND algorithms guess the noise effect sequence $\mathbf{z}$ rather than the real valued noise $\mathbf{n}$. Nonetheless, in ORBGRAND $\tilde{\mathbf{y}}$ is used to extract soft channel information to assess the reliability of each hard decision demodulated symbol of $\mathbf{y}$ leading to improved decoding performance. For the sake of simplicity, Binary Phase-Shift Keying (BPSK) modulation is assumed in the following.

With C being the code-book of the given coding scheme, and $f_{\tilde{\mathbf{y}}|C}$ being the probability density function of $\tilde{\mathbf{y}}$ given C , the log-likelihood ratio is defined as

$$LLR(\tilde{y}_j) = \log \left(\frac{f_{\tilde{\mathbf{y}}|C}(\tilde{y}_j|1)}{f_{\tilde{\mathbf{y}}|C}(\tilde{y}_j|0)} \right) \forall j = 1, 2, \dots, n \quad (2.2)$$

Demodulation of $\tilde{y}_j$ into y_j for BPSK is achieved through $y_j = (\text{sign}(\tilde{y}_j) + 1)/2$, and $|LLR(\tilde{y}_j)|$ is referred to as the *reliability* of y_j .

To quantitatively capture the soft information in $\tilde{\mathbf{y}}$, ORBGRAND computes the *a posteriori* likelihood that the hard decision demodulated bit y_j is in error, which can be expressed in terms of the bit reliabilities as:

$$B_j = \frac{e^{-|LLR(\tilde{y}_j)|}}{1 + e^{-|LLR(\tilde{y}_j)|}} \in [0, 0.5] \quad (2.3)$$

2 Theory

where B_j is monotonically decreasing with increasing reliability $|LLR(\tilde{y}_j)|$. The resulting vector $\mathbf{B} = (B_1, B_2, \dots, B_n)$ can then be used to evaluate the *a posteriori* likelihood of a binary noise-effect sequence $\mathbf{z}$:

$$\begin{aligned} P(\mathbf{Z} = \mathbf{z}) &= \prod_{j:z_j=0} (1 - B_j) \prod_{j:z_j=1} B_j = \prod_{j=1}^n (1 - B_j) \prod_{j:z_j=1} \frac{B_j}{1 - B_j} \propto \prod_{j:z_j=1} \frac{B_j}{1 - B_j} \\ &= \exp \left(- \sum_{j=1}^n |LLR(\tilde{y}_j)| z_j \right) \quad (2.4) \end{aligned}$$

It follows that up to a common constant, the likelihood of any given putative noise effect sequence $\mathbf{z}$ is determined by the sum of the reliabilities of hard detected bits being flipped, $Rel(\mathbf{z}) = \sum_{j=1}^n |LLR(\tilde{y}_j)| z_j$. This implies that ordering putative noise effect sequences $\mathbf{z}$ in decreasing probability order as required by soft information incorporating GRAND algorithms, amounts to ordering them by ascending reliability sum $Rel(\mathbf{z})$. ORBGRAND introduces a more relaxed ordering, leading to desirable algorithmic properties that are amenable to hardware implementation, as detailed in the following sections.

2.2.3 Reliability weight

For notational simplicity, incoming hard detected symbols y_j of $\mathbf{y}$, are assumed to arrive in order of increasing reliability, such that $|LLR(\tilde{y}_{j_1})| < |LLR(\tilde{y}_{j_2})| \forall j_1 < j_2$. In practice this is achieved by sorting each received $\tilde{y}_j$ accordingly, and requires the original order to be registered as to allow for reversion to the original at decoding completion. The core approach underlying ORBGRAND is the development of statistical models of this ordered $|LLR(\tilde{y}_j)|$ sequence that are accurate, robust and lead to computationally efficient algorithms for generating appropriately ordered noise effect sequences.

This ordered reliability sequence, which will be referred to as *reliability curve*, is approximated in a piece-wise linear fashion. From [9], it is known that a closer approximation of the reliability curve, obtainable by choosing a larger number m of approximation segments, yields better decoding performance, but is coupled with an increase in required computational effort. Furthermore the returns in decoding performance as m increases eventually stagnate, as is illustrated in the plot 5.1a. In general, at low energy per bit to noise power spectral density ratio ($\frac{Eb}{N0}$) values, the reliability curve exhibits linear tendencies, which allows a low number m of segments to yield a good approximation, resulting in good decoding performance, while at higher $\frac{Eb}{N0}$ values the reliability curve becomes increasingly non-linear, which leads to ORBGRAND's decoding performance to benefit from a more numerous segmentation. This behaviour, along with the associated piecewise approximation, can be seen in figures 2.1a and 2.1b.

2 Theory

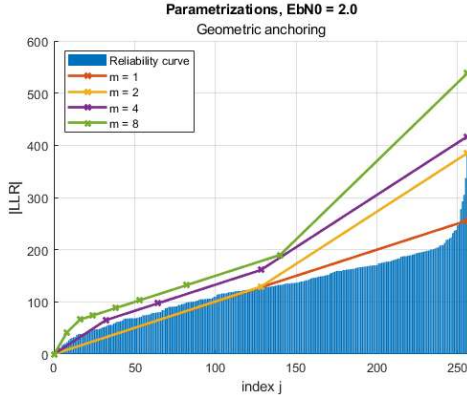
To find the defining parameters of the approximation segments, multiple approaches for either higher accuracy or lower complexity are possible [9]. In general the indexes of the n reliability-ordered input data bits are divided into m not necessarily equal consecutive sections, each denoted by the suffix i and spanning from index $I_{i-1} + 1$ to I_i , where $I_i \in \{1, 2, \dots, n\}$ are named *anchoring points* in this work. These anchoring points are then used to calculate the normalized discretized slope β_i and the normalized discretized offset J_{i-1} of each segment in the approximation. Note that it is crucial for these parameters to be integers, as this enables efficient algorithmic implementations of putative noise effect sequences generation [9]. A valid approach, assuming $m > 1$ is detailed here:

$$\text{Let } \eta_i = \begin{cases} \frac{|LLR(\tilde{y}_{I_2})| - |LLR(\tilde{y}_1)|}{I_2 - 1} & \text{if } i = 1 \\ \frac{|LLR(\tilde{y}_{I_i})| - |LLR(\tilde{y}_{I_{i-1}})|}{I_i - I_{i-1}} & \text{if } i = 2, 3, \dots, m \end{cases} \quad \text{and} \quad (2.5)$$

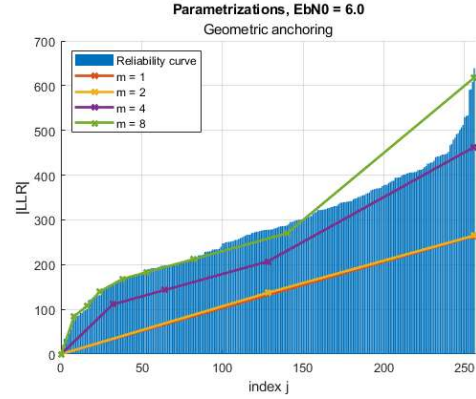
$$Q = \arg \min_{i=1,2,\dots,m} \{\eta_i\} \quad (2.6)$$

$$\beta_i = \left\lfloor \frac{\eta_i}{Q} \right\rfloor \quad \text{for } i = 1, 2, \dots, m \quad \text{and} \quad (2.7)$$

$$J_i = \begin{cases} \left\lfloor \frac{|LLR(\tilde{y}_1)|}{Q} \right\rfloor & \text{if } i = 0 \\ \left\lfloor J_{i-1} + \beta_i(I_i - I_{i-1}) \right\rfloor & \text{if } i = 1, 2, \dots, m-1 \end{cases} \quad (2.8)$$



(a) Geometric anchoring Eb/N0 = 2 [dB]



(b) Geometric anchoring Eb/N0 = 6 [dB]

Figure 2.1: Reliability curve approximations of a data vector of length $n = 256$ for different Eb/N0 values, scaled to allow a better illustration.

From the piecewise approximation of the reliability curve an approximate measure of the reliability of the ordered received symbols can be defined:

2 Theory

$$|LLR(y_j)| \approx \lambda_j = J_{i-1} + \beta_i(j - I_{i-1}) \text{ for } I_{i-1} < j \leq I_i \text{ where } i \text{ is the segment index.} \quad (2.9)$$

From the approximate symbol reliability λ_j , the approximate reliability sum of any $\mathbf{z}$, which shall be called *reliability weight* $W_{rel}(\mathbf{z})$ of the noise effect sequence, can be found as shown below.

$$\begin{aligned} Rel(\mathbf{z}) &= \sum_{j=1}^n |LLR(\tilde{y}_j)| z_j \approx W_{rel}(\mathbf{z}) = \sum_{j=1}^n \lambda_j z_j = \sum_{i=1}^m \sum_{j=I_{i-1}+1}^{I_i} \lambda_j z_j \\ &= \sum_{i=1}^m \sum_{j=I_{i-1}+1}^{I_i} J_{i-1} z_j + \sum_{i=1}^m \sum_{j=I_{i-1}+1}^{I_i} \beta_i(j - I_{i-1}) z_j \\ &= \sum_{i=1}^m J_{i-1} \sum_{j=I_{i-1}+1}^{I_i} z_j + \sum_{i=1}^m \beta_i \sum_{j=I_{i-1}+1}^{I_i} (j - I_{i-1}) z_j \\ &= \sum_{i=1}^m J_{i-1} w_H(\mathbf{z}_i) + \sum_{i=1}^m \beta_i w_L(\mathbf{z}_i) \in \mathbb{Z}_{\geq 0} \end{aligned} \quad (2.10)$$

Where $\mathbf{z}_i$ denotes the noise effect sub-sequence associated to section i , $w_H(\mathbf{z}_i)$ its Hamming weight or number of non-zero bits within $\mathbf{z}_i$, and $w_L(\mathbf{z}_i)$ the logistic weight or sum of the relative indexes $j - I_{i-1}$ of non-zero bits within $\mathbf{z}_i$.

2.2.4 Noise effect sequences generation

To yield near soft-ML decoding performance [15], ORBGRAND generates noise effect sequences in ascending reliability weight ordering. For any given reliability weight W_{rel} multiple associated noise effect sequences can exist, which can then be subtracted from the hard detected datavector $\mathbf{y}$ and code-book checked in arbitrary order. In this work, each putative noise effect sequence trial represents an iteration performed by an ORBGRAND decoder. To find the set S_W of all $\mathbf{z}$ such that $W_{rel}(\mathbf{z}) = W$, the reliability weight W has to be partitioned amongst the m intervals. There could be many possible partitions, all of which have to be found. This reduces to a possibly repeating, non-negative, integer partition problem, efficiently solvable through Algorithm 2. Each partition dictates how much each of the m partial noise sequences $\mathbf{z}_i$ contributes in terms of partial reliability weight, denoted by W_i , to the total of W . Each partition will be referred to as *splitting pattern* in this work and denoted by $\xi = \{W_1, W_2, \dots, W_m\}$, while the set of all splitting patterns of a certain reliability weight W , will be denoted by $\Xi_W = \{\xi \in \mathbb{Z}_{\geq 0}^m : \sum_{i=1}^m W_i = W\}$.

Algorithm 2: The Integer Splitting Algorithm

Input: W, m
Output: Ξ_W

```

1  $k \leftarrow 0$ ;
2 for  $W_1 = 0$  to  $W$  do
3   for  $W_2 = 0$  to  $W - W_1$  do
4      $\vdots$  (nested loops over  $W_i, i = 3, 4, \dots, m-2$ );
5     for  $W_{m-1} = 0$  to  $W - \sum_{l=1}^{m-2} W_l$  do
6        $W_m \leftarrow W - \sum_{l=1}^{m-1} W_l$ ;
7        $k \leftarrow k + 1$ ;
8        $\xi \leftarrow \{W_1, W_2, \dots, W_m\}$ ;
9        $\Xi_W \leftarrow \xi$ ;
10    end
11  end
12 end
13  $\vdots$  (nested loops over  $W_i, i = 3, 4, \dots, m-2$ );
14 return  $\Xi_W$ ;
```

For each $W_i \in \xi$, associated to interval i and for each $\xi \in \Xi_W$, multiple partial noise effect sequences $\mathbf{z}_i$ s.t. $W_{rel}(\mathbf{z}_i) = W_i$ can be found, the set of which is denoted as $\Psi_{W_i} = \left\{ \mathbf{z}_i : \sum_{j=I_{i-1}+1}^{I_i} \lambda_j z_j = W_i \right\}$.

From equation (2.10) follows that each $\mathbf{z}_i \in \Psi_{W_i}$ will satisfy:

$$W_i = J_{i-1}w_H(\mathbf{z}_i) + \beta_i w_L(\mathbf{z}_i) \Leftrightarrow w_L(\mathbf{z}_i) = \sum_{k=1}^{w_H(\mathbf{z}_i)} v_k = \frac{W_i - w_H(\mathbf{z}_i)J_{i-1}}{\beta_i} \quad (2.11)$$

where v_k stands for the relative index of the non-zero bits $z_{I_{i-1}+k}$ of the partial noise effect sequence. Considering that $w_L(\mathbf{z}_i) \in \mathbb{Z}_{\geq 0}$, it follows from equation (2.11) that $W_i - w_H(\mathbf{z}_i)J_{i-1}$ has to be divisible by β_i . Furthermore, the lower bound for the possible values of $w_L(\mathbf{z}_i)$ for all sections i where $W_i \neq 0$, is reached by the partial noise effect sequence with all $w_H(\mathbf{z}_i)$ non-zero bits situated consecutively at the lowest relative indexes $j \in \{1, 2, \dots, w_H(\mathbf{z}_i)\}$. This minimum partial logistic weight, will amount to the sum of all integers up to and including $w_H(\mathbf{z}_i)$ and can be calculated by the Euler Summation Formula. The upper bound for the partial logistic weight is reached instead by the partial noise effect sequence $\mathbf{z}_i$ where all $w_H(\mathbf{z}_i)$ non-zero bits are positioned consecutively at the largest possible relative indexes $j \in \{I_i - I_{i-1} - w_H(\mathbf{z}_i), I_i - I_{i-1} - w_H(\mathbf{z}_i) + 1, \dots, I_i - I_{i-1}\}$. The derivation of the formula for both the upper bound and the lower bound of the partial logistic weight $w_{L_i} = w_L(\mathbf{z}_i)$ given a partial Hamming weight $w_{H_i} = w_H(\mathbf{z}_i)$ for any partial noise effect sequence $\mathbf{z}_i$, is reported in equations (2.12) and (2.13). The

2 Theory

structure of the partial noise-effect sequences resulting in these bounds is reported in (2.12) for the special case of $I_i - I_{i-1} = 8$ and Hamming weight $w_{H_i} = 4$.

Example of upper and lower bounds:

$$w_{L_{\min}} : [1, 1, 1, 1, 0, 0, 0, 0, 0] \quad w_{L_{\max}} : [0, 0, 0, 0, 0, 1, 1, 1, 1] \quad (2.12)$$

$$w_{L_i} \geq \sum_{k=1}^{w_{H_i}} = \frac{w_{H_i}(1 + w_{H_i})}{2} \quad (2.13)$$

Let $\gamma_i = I_i - I_{i-1}$ denote the number of indexes in the interval i

$$\begin{aligned} w_{L_i} &\leq \frac{\gamma_i(\gamma_i + 1)}{2} - \frac{(\gamma_i - w_{H_i})(\gamma_i - w_{H_i} + 1)}{2} \\ &= \frac{2\gamma_i w_{H_i} + w_{H_i} - w_{H_i}^2}{2} \\ &= \gamma_i w_{H_i} - \frac{w_{H_i}(w_{H_i} - 1)}{2} \\ &= \gamma_i w_{H_i} + w_{H_i} - \frac{w_{H_i}(w_{H_i} + 1)}{2} \\ &= (I_i - I_{i-1} + 1)w_{H_i} - \frac{w_{H_i}(w_{H_i} + 1)}{2} \end{aligned} \quad (2.14)$$

Combining all conditions yields the following:

$$\begin{cases} W_i = 0 \text{ or } W_i - w_{H_i} J_{i-1} \geq \beta_i \frac{w_{H_i}(w_{H_i} + 1)}{2} \\ W_i - w_{H_i} J_{i-1} \leq \beta_i (I_i - I_{i-1} + 1)w_{H_i} - \beta_i \frac{w_{H_i}(w_{H_i} + 1)}{2} \\ W_i - w_{H_i} J_{i-1} \text{ is divisible by } \beta_i \end{cases} \quad (2.15)$$

For a given W_i , pseudo-code to find the set H_{W_i} of all partial Hamming weights w_{H_i} satisfying condition (2.15), can be found through Algorithm 3. As conditions (2.15) are not guaranteed to be achievable for any given W_i , it is possible that Algorithm 3 fails, which invalidates the whole splitting pattern ξ . In the case where all the splitting patterns $\xi \in \Xi_W$ are invalidated, the current reliability weight W is deemed impossible, and ORBGRAND will continue with $W_{\text{new}} = W + 1$.

Considering a single partial reliability weight W_i , associated to a valid Hamming weight $w_{H_i} \in H_{W_i}$ and logistic weight w_{L_i} . Equation (2.11) shows that computing all non repeating positive integer partitions of w_{L_i} yields all possible combinations of relative indexes v_k associated to non-zero noise effect bits, which can be translated into the desired

Algorithm 3: The Collection Algorithm for Valid Partial Hamming Weights

Input: W_i, J_{i-1}

Output: H_{W_i} or FAIL

```

1  $k \leftarrow 0$ ;
2 for  $w_{H_i} = 1$  to  $\left\lfloor \frac{\sqrt{1+8W_i-1}}{2} \right\rfloor$  do
3   if  $W_i, w_{H_i}$  and  $J_{i-1}$  satisfy (2.15) then
4      $k \leftarrow k + 1$ ;
5      $H_{W_i} \leftarrow w_{H_i}$ ;
6   end
7 end
8 if  $k = 0$  then
9   return FAIL;
10 end
11 else
12   return  $H_{W_i}$ ;
13 end

```

partial noise effect sequences $\{\mathbf{z}_i \in \{0, 1\}^{I_i - I_{i-1}} \text{ s. t. } w_L(\mathbf{z}_i) = w_{L_i} \text{ and } w_H(\mathbf{z}_i) = w_{H_i}\}$. This step can be efficiently achieved through the *Landslide* algorithm presented in section 2.2.5. Repeating this step $\forall w_{H_i} \in H_{W_i}$, yields the set Ψ_{W_i} , which contains all partial noise effect sequences $\mathbf{z}_i$, of partial reliability weight W_i .

From the obtained sets $\Psi_{W_i} \forall i = 1, 2, \dots, m$, the set of all noise effect sequences associated to a certain splitting pattern ξ is given by their cartesian product. The total set of putative noise effect sequences of a given reliability weight S_W of all $\mathbf{z}$ such that $W_{rel}(\mathbf{z}) = W$, is then given by the union of the cartesian products amongst all sets Ψ_{W_i} where $i = 1, 2, \dots, m$ over all splitting patterns $\xi \in \Xi_W$. This is shown in equation (2.16), where $\times$ marks the Cartesian product amongst the sets Ψ_{W_i} .

$$S_W = \left\{ \mathbf{z} \in \{0, 1\}^n : \sum_{i=1}^m \sum_{j=I_{i-1}+1}^{I_i} \lambda_j z_j = W \right\} = \bigcup_{\xi: \sum_{i=1}^m W_i = W} (\Psi_{W_1} \times \Psi_{W_2} \times \dots \times \Psi_{W_m}) \quad (2.16)$$

As all putative noise effect sequences $\mathbf{z} \in S_W$ are by definition of identical reliability weight W , their probability is deemed identical by ORBGRAND. This means that all $\mathbf{z}$ of a given S_W are subtracted from the hard detected data vector $\mathbf{y}$ and subsequently code-book checked, in arbitrary order. If all code-book checks fail, new noise effect sequences of reliability weight $W_{new} = W + 1$ are generated continuing the iterative process, until a valid decoding is found.

2.2.5 Landslide algorithm

In section 2.2.4 it is shown, that the generation of partial noise effect patterns $\mathbf{z}_i$ of interval i , given a partial reliability weight W_i and a Hamming weight $w_H(\mathbf{z}_i) = w_{H_i}$, is equivalent to finding all non-repeating positive integer partitions of the associated logistic weight $w_L(\mathbf{z}_i) = w_{L_i}$:

$$\begin{aligned} & \text{Let } \gamma_i = I_i - I_{i-1}, \text{ be the number of bits in interval } i \\ & \{\mathbf{z}_i \in \{0, 1\}^{\gamma_i} : W_i = J_{i-1}w_{H_i} + \beta_i w_{L_i}, w_H(\mathbf{z}_i) = w_{H_i}\} \\ & = \left\{ v^k \in \mathbb{N}^{w_{H_i}} : 1 \leq v_1 < v_2 < \dots < v_{w_{H_i}} \leq \gamma_i, \sum_{k=1}^{w_{H_i}} v_k = w_{L_i} \right\} \end{aligned} \quad (2.17)$$

Defining $v_k = u_k + k \Leftrightarrow u_k = v_k - k$ for $k = 1, 2, \dots, w_{H_i}$, permits formulating the problem as computing the set:

$$\begin{aligned} & \text{Let } \overline{\gamma}_i = \gamma_i - w_{H_i} \text{ and } \overline{w}_{L_i} = w_{L_i} - \frac{w_{H_i}(w_{H_i} + 1)}{2} \\ & \left\{ u^{w_{H_i}} \in \mathbb{Z}_{\geq 0}^{w_{H_i}} : 0 \leq u_1 \leq u_2 \leq \dots \leq u_{w_{H_i}} \leq \overline{\gamma}_i, \sum_{k=1}^{w_{H_i}} u_k = \overline{w}_{L_i} \right\} \end{aligned} \quad (2.18)$$

Which corresponds to finding all integer partitions of $\overline{w}_{L_i}$ of size w_{H_i} , composed of possibly repeating, non-negative integers $u_k \in \{0, 1, \dots, \overline{\gamma}_i\}$. This task can be executed efficiently by the *Landslide* algorithm as first introduced in [9], whose pseudo-code is reported in Algorithm 4. As explained in [9], this integer splitting algorithm is highly parallelizable and lends itself to hardware implementation.

In the quest of detailing the Landslide algorithm, it can be advantageous to represent each partition as a mirrored Ferrers Diagram, as illustrated in figure 2.2. Each column of the diagrams represents an integer part of value u_k , equaling the number of cells in the column and the total number of cells in each diagram equaling the integer $\overline{w}_{L_i} = \sum_{k=1}^{\overline{\gamma}_i} u_k$ to be partitioned.

Crucial to the execution of this algorithm is function (2.19), which records the "drop" between adjacent $k, k + 1$ parts:

$$d : \{1, 2, \dots, w_{H_i}\} \mapsto \{0, 1, \dots, \overline{w}_{L_i}\}; \quad d(k) = \begin{cases} 0 & \text{if } k = w_{H_i} \\ u_{k+1} - u_k & \text{if } k \in \{1, 2, \dots, w_{H_i} - 1\} \end{cases} \quad (2.19)$$

2 Theory

From (2.19) the accumulated drop $D(k) = \sum_{f=k}^{w_{H_i}} d(f)$ is defined, where $D(1)$ records the total drop in the integer partition $u^{w_{H_i}} = \{u_1, u_2, \dots, u_k\}$, of $\overline{w_{L_i}}$.

The Landslide algorithm heavily relies on the Build-Mountain routine, which forms partitions by pushing unallocated cells to the right-most parts, akin to building the steepest, highest mountain allowable on the right side of the diagram. In general, when the values of $\{u_1, u_2, \dots, u_l\}$ have been specified, the allocation of the remaining cells $\{u_{l+1}, u_{l+2}, \dots, u_{w_{H_i}}\}$ is performed by the Build-Mountain routine as follows:

1. $u_k \leftarrow u_l$ for $l + 1 < k < w_{H_i}$
2. $\overline{w_{L_i}} \leftarrow \overline{w_{L_i}} - \sum_{i=1}^{w_{H_i}} u_i$
3. Obtain q and r such that $\overline{w_{L_i}} = q(\overline{\gamma_i} - u_k) + r$
4. if $q \neq 0$, $u_i \leftarrow \overline{\gamma_i}$, for $w_{H_i} - q + 1 \leq i \leq w_{H_i}$
5. $u_{w_{H_i}-q} \leftarrow u_{w_{H_i}-q} + r$

The initial partition is obtained analogously by assuming a dummy partition $u_0 = 0$.

Algorithm 4: The Landslide Algorithm

Input: $\overline{w_{L_i}}, w_{H_i}, \overline{\gamma_i}$

Output: Set U of all possible $u^{w_{H_i}}$

```

1 Build-mountain for initial partition;
2  $j \leftarrow 1$ ;
3  $U \leftarrow u^{w_{H_i}}$ ;
4 Update  $D(i)$  for  $1 \leq i \leq w_{H_i}$ ;
5 while  $D(1) \geq 2$  do
6   | Locate the largest  $k$  such that  $D(k) \geq 2$ ;
7   |  $u_k \leftarrow u_k + 1$ ;
8   | Build-mountain from  $u_k$ ;
9   | Update  $D(i)$  for  $1 \leq i \leq w_{H_i}$ ;
10  |  $j \leftarrow j + 1$ ;
11  |  $U \leftarrow u^{w_{H_i}}$ ;
12 end
13 return  $U$ ;
```

Once all possible integer partitions of $\overline{w_{L_i}}$ of size w_{H_i} , composed of possibly repeating, non-negative integers of maximum size $\overline{\gamma_i}$ have been computed, they are reverted to $v_k = u_k + k \forall k \in \{1, 2, \dots, w_{H_i}\}$, which can finally be used to generate the associated partial noise effect sequences as described in section 2.2.4.

The following numerical example aims at explaining the workings of the Landslide algorithm in finding a partial noise effect sequence:

2 Theory

Suppose $i = 2$, s. t. $J_1 = 8, \beta_2 = 2, I_1 = 8, I_2 = 16 \Rightarrow \gamma_2 = I_2 - I_1 = 8$
 objective is to find the set $\mathbf{Z}_2$ of all $\mathbf{z}_2 \in \{0, 1\}^8$
 s. t. $w_L(\mathbf{z}_2) = 20 =: w_{L_2}$ and $w_H(\mathbf{z}_2) = 5 =: w_{H_2}$
 $\overline{\gamma_2} = \gamma_2 - w_{H_2} = 3$ and $\overline{w_{L_i}} = w_{L_2} - \frac{w_{L_2}(w_{L_2} + 1)}{2} = 5$ (2.20)

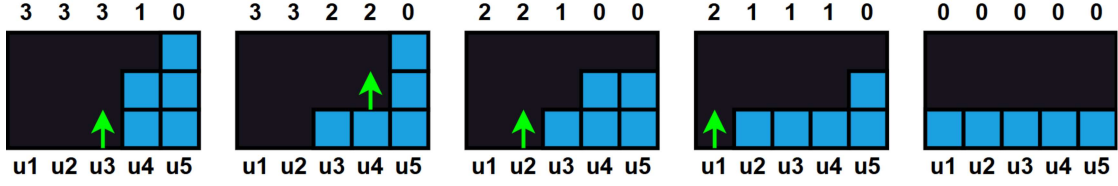


Figure 2.2: Ferris diagram of Landslide algorithm. The upward arrow indicates the corresponding part is to be increased by 1 in the next step.

$$U = \{\{0, 0, 0, 2, 3\}, \{0, 0, 1, 1, 3\}, \{0, 0, 1, 2, 2\}, \{0, 1, 1, 1, 2\}, \{1, 1, 1, 1, 1\}\}$$

Using that $v_k = u_k + k$ the following is obtained:

$$V = \{\{1, 2, 3, 6, 8\}, \{1, 2, 4, 5, 8\}, \{1, 2, 4, 6, 7\}, \{1, 3, 4, 5, 7\}, \{2, 3, 4, 5, 6\}\}$$

further, using that v_k is the relative index of non-zero entries of $\mathbf{z}_2$

The desired set is: $\mathbf{Z}_2 = \{\{1, 1, 1, 0, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1, 0, 0, 1\},$
 $\{1, 1, 0, 1, 0, 1, 1, 0\}, \{1, 0, 1, 1, 1, 0, 1, 0\}, \{0, 1, 1, 1, 1, 1, 0, 0\}\}$ (2.21)

2.3 FIFO-Scheduling

2.3.1 Motivation

GRAND algorithms decode data vectors by means of iterative putative noise-effect sequence subtraction from the received hard detected datavector $\mathbf{y}$, followed by a code-book membership test on the resulting bitstring, as detailed in sections 2.1 and 2.2. The first codeword found in this manner is returned as the decoded datavector. Datavector corruption through noise is fundamentally random, which leads to the observation that guessing the actual noise-effect sequence as done in GRAND calls for a nondeterministic number of iterations. This inconsistent runtime makes GRAND decoders unfit for direct insertion into real world systems, where strictly constant throughput is required.

In this work FIFO-Scheduling is explored as a solution to this limitation of ORB-GRAND. This approach, first introduced in [20], relies on input buffering, parallel de-

coding and output reordering, providing a flexible latency environment which allows nondeterministic decoding schemes such as GRAND to operate in fixed throughput systems.

2.3.2 Structure

The general structure underlying FIFO-Scheduling, as detailed in the following, consists of an input first-in first-out (FIFO) buffer, which feeds into several parallel decoding units through a distribution unit. The resulting decodings are released through a collection unit into a re-ordering buffer at the output, from which the data is gradually outputted from the system in original input ordering. An overview of the general structure of a FIFO-Scheduling system, including the early termination mechanism detailed in section 2.3.3 and the implementation specific booking mechanism explained in section 4.2.4, is provided in figure 2.3.

As received hard demodulated datavectors $\mathbf{y}$ are inputted into any FIFO-Scheduling architecture at a regular rate I , they are first stowed in a FIFO, whose number of slots will be referred to as F . The FIFO buffers the regularly incoming $\mathbf{y}$, from the irregular throughput of the GRAND decoders, allowing to take greater advantage of the decoding capacity of the system. This is possible as for one, if the decoders happen to be busy when a new $\mathbf{y}$ enters the system, it is queued in the FIFO and not lost. Secondly, provided the FIFO is not empty, as soon as a decoder terminates and is thus able to accept new data, it is immediately provided with a fresh data vector and put to work, without having to wait for the upcoming data vector.

As soon as the system is ready to decode the next data vector, which implies that an ORBGRAND unit is able to accept a fresh input, the FIFO releases the next $\mathbf{y}$, maintaining the original input ordering. This data vector is assigned to a free decoder by means of a distribution unit, which starts the nondeterministic iterative decoding process. The number of decoders employed by the FIFO-Scheduling system will be termed D . Each time a decoder terminates, it releases the found codeword through a collection unit into a re-ordering buffer. This output buffering is justified by the nondeterministic decoding latency inherent to GRAND algorithms, which causes that the distributed data vectors to leave the decoders in arbitrary order. The re-ordering buffer is therefore responsible for storing the decodings until all preceding decodings are found, then releasing them in the original order.

2.3.3 Early Termination

FIFO-Scheduling stowes the regularly incoming data vectors $\mathbf{y}$ within the FIFO, and releases them in the distribution unit as soon as possible. The exact conditions that have to be met for the FIFO to be able to pass data vectors further into the system depend on the implementation and one possible approach will be discussed in detail in

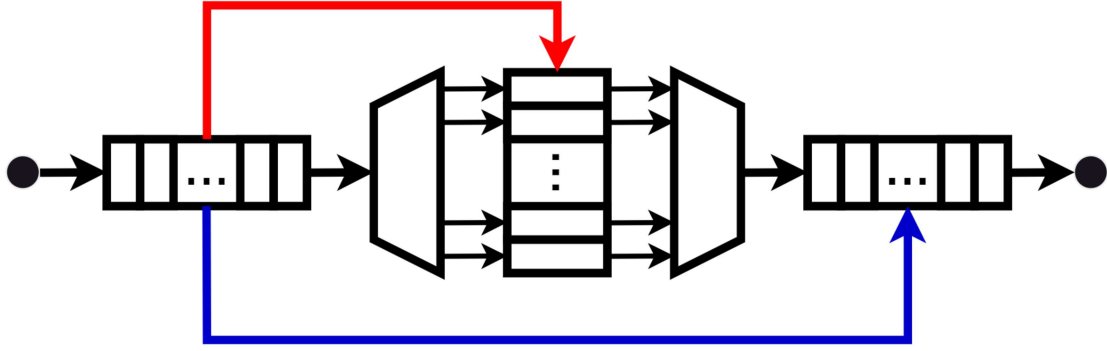


Figure 2.3: FIFO-Scheduling Structure, red arrow: Early termination mechanism, blue arrow: Booking mechanism

section 4.2.1. Crucially, the FIFO is only able to release data vectors if a free decoder is available, which paired with the nondeterministic decoding latency inherent to GRAND decoders, causes the FIFO releases to be irregular. This leads to situations, where the FIFO is fully occupied leading to the system to be unable to accept further data vectors. Dataloss happens if this situation persists as new $\mathbf{y}$ arrive, which calls for an appropriate solution. In this work a variant of the early termination mechanism proposed in [20] is employed, which consists in terminating the longest running decoding process, flushing the associated decoding unit, and thus allowing the FIFO to release a data vector into the distribution unit. This leads to an available slot within the FIFO, mitigating possible dataloss at the next $\mathbf{y}$ arrival. The flow chart of a general early termination mechanism is given in figure 2.4.

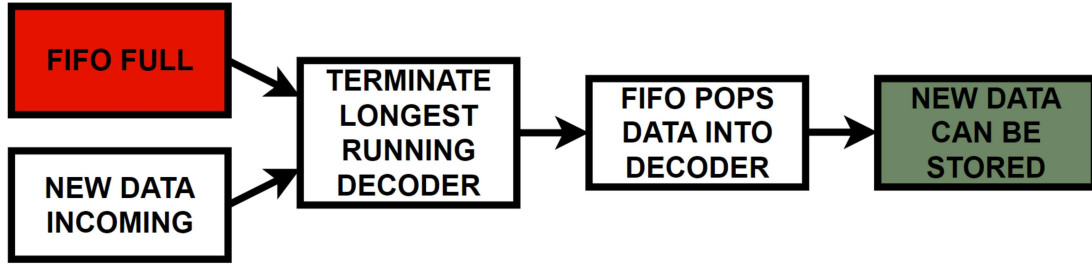


Figure 2.4: Early termination flow chart

ORBGRAND Simulation

3.1 Overview

Transmission of a codeword $\mathbf{x}$ modulated into a symbol vector $\tilde{\mathbf{x}}$ over a real-world communication channel, results in noisy data at the receiver. Modeling the noise as AWGN results in the real-valued received symbol vector $\tilde{\mathbf{y}} = \tilde{\mathbf{x}} + \mathbf{n}$, where $\mathbf{n}$ stands for the real valued noise. Hard detection subsequently yields the data vector $\mathbf{y} = \mathbf{x} \oplus \mathbf{z}$, where $\mathbf{z}$ represents the binary noise effect sequence corrupting the transmitted codeword $\mathbf{x}$. Any GRAND algorithm aims at guessing this noise effect sequence, by iteratively subtracting putative $\mathbf{z}$ from $\mathbf{y}$ and performing a code-specific code-book membership test on the resulting $\hat{\mathbf{x}} = \mathbf{y} \oplus \mathbf{z}$. ORBGRAND additionally incorporates soft decoding information in the decoding process, resulting in near soft-ML decoding performance, while preserving parallelism and suitability for hardware implementation [9]. Near soft-ML decoding performance requires the putative noise effect sequences to be guessed in decreasing likelihood. An approximation of this ordering is achieved by defining the reliability weight W_{rel} of a given sequence. The reliability weight of a given noise effect sequence $\mathbf{z}$ is an approximation of the cumulative reliability $Rel(\mathbf{z}) = \sum_{j=1}^n |LLR(\tilde{y}_j)| z_j$ of all n received bits of the data vector $\mathbf{y}$, which is inversely proportional to its likelihood. This means that ORBGRAND iteratively generates and tests noise sequences in increasing W_{rel} ordering.

The implementation of ORBGRAND in this work, closely follows the algorithm description given in [9] and is detailed in the following.

3.2 Reliability curve approximation

In ORBGRAND, to make use of the received soft detection information, the bits of $\mathbf{y}$ are firstly ordered in ascending reliability $|LLR(\tilde{y}_i)|$. The original input ordering is stored in an index vector, to be restored at the end of the process. A segmentwise approximation of the resulting reliability curve is then employed through which the reliability weight $W_{\text{rel}}(\mathbf{z})$ of any putative noise-effect sequence $\mathbf{z}$ is defined, as explained in section 2.2.3. In this work, the defining parameters I_{i-1} , I_i , β_i and J_{i-1} of each of the m approximation segments are computed as described in the following.

As a first step, the indexes of the n reliability ordered input data bits are divided into m consecutive sections, each denoted by the suffix i and spanning from bit index $I_{i-1} + 1$ to I_i . A constant spacing of the section delimiters I_i will be termed *linear anchoring*, while sectioning where partition sizes double for each subsequent section akin to a geometric series, will be termed as *geometric anchoring*, the calculation of which is detailed in equation 3.1.

$$\begin{array}{cc}
 \text{Linear anchoring} & \text{Geometric anchoring} \\
 I_i = \begin{cases} 0 & \text{if } i = 0 \\ I_{i-1} + \lfloor \frac{n}{m} \rfloor & \text{if } i = 1, 2, \dots, m \\ n & \text{if } i = m + 1 \end{cases} & I_i = \begin{cases} 0 & \text{if } i = 0 \\ \lfloor \frac{n}{2^{m+1-i}} \rfloor & \text{if } i = 1, 2, \dots, m \\ n & \text{if } i = m + 1 \end{cases}
 \end{array} \quad (3.1)$$

From from these delimiters I_i , the remaining normalized segment offsets J_i and normalized segment slopes β_i are computed for each i as described in equation (2.5). An illustration of reliability curve approximations based on both linear and geometric anchoring is given in figures 3.1a and 3.1b.

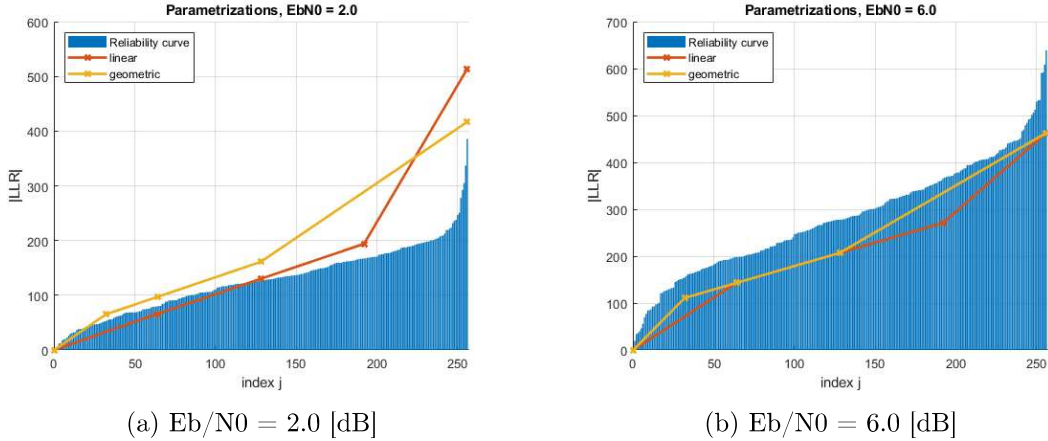


Figure 3.1: Geometric and linear anchoring comparison. Scaled to improve illustration.

3.3 Putative noise generation

ORBGRAND tests noise effect sequences in ascending reliability weight W_{rel} order, which is an approximate measure of the improbability of a given noise effect sequence $\mathbf{z}$ and is derived from the approximation of the reliability curve. Initially, the lowest reliability weight $W = 0$, which corresponds to the all zero noise effect sequence $\mathbf{z} = \{0\}^n$, results in the code-book check $\hat{\mathbf{x}} = \mathbf{y} \oplus \mathbf{z} = \mathbf{y}$. Starting from this trivial situation, ORBGRAND gradually increases the reliability weight by one, iteratively computing and testing all associated noise effect sequences.

From a particular reliability weight W , out of which the set S_W of all noise effect sequences $\mathbf{z} : W_{\text{rel}}(\mathbf{z}) = W$ is to be found, the process begins by finding all possible ways to distribute W into partial reliability weights W_i amongst the m sections. Such partitions are called splitting patterns ξ and are found through Algorithm 2. As potentially many unachievable partitions are found in this way, it is wise to perform some preliminary pruning during the execution of this algorithm, by rejecting any partial reliability weight W_i which does not fulfill the following conditions:

Defining $\gamma_i = I_i - I_{i-1}$

$$\forall i \in \{1, 2, \dots, m\} \text{ if } W_i \neq 0 \Rightarrow \text{it must hold that } J_i + \beta_i \leq W_i \leq \gamma_i J_i + \beta_i \frac{\gamma_i(\gamma_i + 1)}{2} \quad (3.2)$$

Conditions (3.2) are derived from the lower and upper bounds of the possible partial reliability weight W_i for a particular section i . Integrating restriction (3.2) into Algorithm 2, yields the set $\Xi_{W_{\text{pruned}}}$, containing all splitting patterns ξ from which noise effects sequences are to be generated.

For each splitting pattern ξ , where each $W_i \in \xi$ fulfills conditions (3.2), the sets $\Psi_{W_i} \forall i \in \{1, 2, \dots, m\}$ are calculated, where Ψ_{W_i} denotes the set of all partial noise effect sequences $\mathbf{z}_i$ of partial reliability weight W_i . As will be explained in section 3.4, not every W_i results in a set Ψ_{W_i} calculation, as previously computed results are saved.

In the case where Ψ_{W_i} is to be computed, the implementation iterates through each valid partial Hamming weight w_{H_i} , returning all possible partial noise effect sequences $\mathbf{z}_i$ s. t. $w_H(\mathbf{z}_i) = w_{H_i}$. The set of all partial Hamming weights which satisfy conditions (2.15) H_{W_i} is found according to Algorithm 3.

For each non-zero partial Hamming weight $w_{H_i} \in H_{W_i}$, all possible partial noise effect sequences are computed through the Landslide algorithm, as detailed in section 2.2.5. In the case where $w_{H_i} = 0$, the all-zero partial noise effect subsequence $z_i = \{0\}^{\gamma_i}$ is returned.

3 ORBGRAND Simulation

From the calculated sets $\Psi_{W_i} \forall i \in \{1, 2, \dots, m\}$, the set of all noise effect sequences associated to the splitting pattern ξ is constructed by means of cartesian products according to equation (2.16). All $\mathbf{z}$ within this resulting set, are used to find the codeword candidate $\hat{\mathbf{x}} = \mathbf{y} \oplus \mathbf{z}$, which is then code-book checked according to Algorithm 1. This process is repeated for all splitting patterns $\xi \in \Xi_{W_{\text{pruned}}}$, for growing reliability weights W , until a successful code-book check occurs, which will be considered the correct decoding.

3.4 Memoization

As noise pattern generation is a computationally involved process, it is desirable to save results to avoid repeated calculation. This is however not as straightforward as for GRAND, where the ordering of the noise effect sequence trials remains constant over all data vectors $\mathbf{y}$. This is because ORBGRAND integrates soft decoding information into the noise effect pattern generation, resulting in a potentially unique ordering for each detected $\mathbf{y}$. However, it can be observed that for a given $\mathbf{y}$, the same partial reliability weight W_i of a given interval i , could be part of several different reliability weights W and several splitting patterns $\xi \in \Xi_W$. This implies that recalculating the sets of partial noise effect sequences Ψ_{W_i} each time is redundant and thus wasteful. Therefore, memorizing the sets Ψ_{W_i} as they are calculated for the first time within the decoding process for a particular y allows for repeated calculations to be avoided. This approach reduces the computational burden of the simulation and is the method employed in this work.

FIFO Scheduling Simulation

4.1 Overview

Given the random nature of channel noise effects, each codeword will present a different level of corruption and will consequently require a variable amount of iterations by GRAND until a valid decoding is found. The nondeterministic running times of GRAND decoders, prove problematic in real-world applications, where a constant throughput is strictly required.

In this work, FIFO-Scheduling as introduced in [20], is explored as a solution to insulate this unwanted property of GRAND decoders, in the particular case of ORBGRAND, permitting real-world system integration. As mentioned in section 2.3, FIFO-Scheduling is enabled by an input first-in first-out (FIFO) buffer of size F , which stores the data vectors $\mathbf{y}$ arriving at regular input rate I , until the system is ready to process them. The FIFO is followed by a distribution unit which feeds into possibly multiple parallel ORBGRAND decoding units, the number of which is given by parameter D . Given that the decoder array might finish decoding a given data vector before its predecessor, the need of an output re-ordering buffer is justified, the size of which will be denoted by R . This output buffer finally releases the found decodings out of the system, in their original input ordering.

To effectively assess the performance of the FIFO-Scheduling architecture, several parameter combinations (I, F, D, R) have to be benchmarked and compared. Doing so requires a stream of numerous data vectors $\mathbf{y}$ to be fed into the FIFO-Scheduling system of a given parametrization, while registering several performance indicators, as detailed in section 4.3. However, computing ORBGRAND decodings according to section 3, is computationally costly. Furthermore, any given data vector $\mathbf{y}$ decoded by ORBGRAND will require the same number of iterations and yield the same result regardless of the surrounding FIFO-Scheduling configuration, provided no early termination happens. This

4 FIFO Scheduling Simulation

means that real-time decoding of data vectors within the FIFO-Scheduling simulation is redundant and computationally costly.

To overcome this inefficiency, a set of data vectors $\mathbf{y}$ is decoded in a first moment and relevant statistics such as the required iterations and the decoding correctness of each are saved together as attributes of a composite data type referred to as struct. These offline decoded data structs are then passed through each of the FIFO-Scheduling parametrizations (I, F, D, R) , allowing the decoders to be emulated as counters. This approach allows to significantly reduce the computational overhead of each simulation, while preserving its required realism.

The pre-calculated data utilized for all FIFO-Scheduling simulations in this work, is obtained by simulating the transmission of numerous randomly generated bitstrings over a noisy AWGN channel while utilizing BPSK modulation. The data vectors $\mathbf{y}$ are encoded by an RLC error correction code of $k = 234$ information bits and $n = 256$ codeword bits, permitting convenient code-book checking through multiplication with the associated parity-check matrix H . The associated redundancy bits are thus $n - k = 22$, which implies achievable decoding complexity of $2^{22} \approx 5e6$ code-book queries [9]. The decoding is performed by a 4 segment approximation ORBGRAND decoder with geometric anchoring, chosen based on the performance-complexity tradeoff described in section 5.1.

4.2 Throughput simulation

The FIFO scheduling system as described in section 2.3 is simulated through a large execution loop, where each iteration defines a single clock cycle and every module is triggered once per cycle. The simulation emulates hardware behaviour through appropriate module tasks execution ordering within each cycle, as listed below. The system can be configured to any valid combination of E_b/N_0 , input rate I , number of parallel decoders D , number of FIFO slots F and number of re-ordering buffer slots R . A given parametrization is deemed to be valid if the re-ordering buffer slots are at least as numerous as the decoders, if more than one decoder is simulated, which is a condition that will be motivated in section 4.2.4. For any valid parametrization, pre-decoded data structs are released into the system at input rate I measured in cycles per codeword.

The FIFO-Scheduling simulation execution loop:

1. **Re-ordering Buffer**
 - a) Output decoding if possible
 - i. Shift all booked slots to the front
2. **New data vector input**
 - a) Release next pre-decoded data struct into the system

3. **FIFO**

- a) Pop first data struct
 - i. Book re-ordering buffer slot if possible
 - ii. Check if a decoder is free
 - iii. Shift all slots to the front
- b) Push new data struct into FIFO
- c) Issue an early termination if necessary

4. **Decoders**

- a) Collect next decoding from idling decoders
 - i. Unload decoding into booked re-ordering buffer slot
- b) Iterate decoders
 - i. Set finished decoders to idle
 - ii. Set longest running decoder to idle if early termination occurred
- c) Distribute new data struct to a free decoder

5. **Collect output**

- a) Collect decoding released by the re-ordering buffer

4.2.1 **FIFO**

Incoming data structs are directly inserted into a free slot of the FIFO, the availability of which is ensured by the early termination mechanism explained in detail in section 2.3.3. The FIFO fills up from the front to the back, and is implemented as a vector of data structs, with pop and push functionalities. When a decoder is available, the FIFO, which in this implementation also acts as a control unit, will book the next free slot in the re-ordering buffer, as detailed in section 4.2.4. The data vector is then popped into the distribution unit, which inserts it into the free ORBGRAND unit. If no re-ordering buffer slot is available to be booked, the FIFO will not release any codeword and wait for the next iteration. Whenever the FIFO is popped, the data vector at its very front is released, after which all further contents are shifted to the front, emulating the behaviour of a real-world FIFO.

4.2.2 Decoders

As the decoding is performed offline, the decoders are implemented as counters that emulate a predetermined number of noise sequence trials each cycle. For simplicity, it is assumed that this number of trials per cycle remains constant over the simulation period, even though proposed hardware implementations might not ensure this [19]. The total number of performed iterations by any individual decoder is compared at each system cycle against the pre-calculated required number of iterations for the associated data vector, which is passed to the decoder within the data struct. Once the number of required trials is reached, the decoder is set to idle, which causes the decoded bitstring to be unloaded through the collection unit into the previously booked re-ordering buffer slot during the next system cycle. In the rare event that multiple decoders terminate during the same cycle, the collection unit will unload the pending decoded bitstrings at a rate of one per cycle, in descending order of their performed iterations. This ordering mechanism is necessary as to allow the re-ordering buffer to accommodate for new bookings as soon as possible, which is particularly crucial in the case of an early termination, as explained in section 4.2.5.

4.2.3 Re-Ordering buffer

Data vectors released from the decoders into the collection unit, are inserted into their previously booked slot of the re-ordering buffer, where they are stored. In the case that the front-most slot is filled, all consecutively filled slots starting from the front, are released at a rate of one per cycle. At each release, all contents of the Re-Ordering buffer, including the bookings, are pushed to the front while maintaining their relative ordering, similarly as in the FIFO. These released decoded data items are ensured to be in their original ordering, as explained in section 4.2.4.

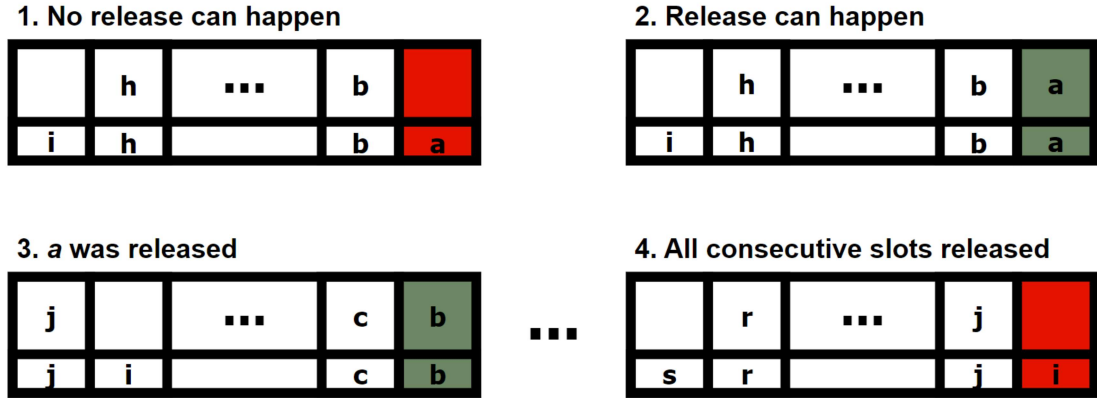


Figure 4.1: Re-ordering buffer in action

4.2.4 Booking mechanism

The parallel decoding units array within the FIFO-Scheduling system, combined with the nondeterministic decoding latency inherent to ORBGRAND decoders, can cause data vectors to be processed in arbitrary order, resulting in the loss of the original data sequence. To mitigate this problem, the input ordering into the system has to be registered at the input, and restored at the output, which in this work is achieved by means of a re-ordering buffer enabled by a booking mechanism.

As data vectors $\mathbf{y}$ enter the FIFO-Scheduling system, they are stored within the FIFO, where their ordering is maintained by the FIFO's filling order, as detailed in 4.2.1. Before being released into a free decoder through the distribution unit, the FIFO, which in this implementation doubles as a control unit, reserves the first not booked slot starting from the front within the re-ordering buffer, action termed *booking* in this work. This means that every data vector being decoded within the system has an associated slot in the re-ordering buffer assigned according to the input ordering. Once data items finish decoding, they are inserted into their booked slot, and once they are part of a consecutively filled section starting from the front, they are released one by one, which will naturally occur in the wanted input ordering. The steps of this process are illustrated in figure 4.2.

From the stringent requirement of data vectors $\mathbf{y}$ to have a booked slot within the re-ordering buffer before being able to be popped from the FIFO and decoded, it follows that having a number of re-ordering buffer slots R smaller than the number of decoders D leads to $D - R$ decoders to not be usable. As simulating a parametrization (I, F, D, R) of the system is computationally expensive, it is useful to exclude parameter combinations which can be classified as ineffective. This motivates the decision to mark as invalid and consequently avoiding the simulation of parametrizations where $R < D$ if $D \geq 1$. For single decoder systems, the Re-Ordering buffers are not necessary, as input ordering is naturally preserved, exceptionally allowing for $D > R$ if $D = 1$. In fact for $D = 1$, only parametrizations where $R = 0$ are considered.

4.2.5 Early termination mechanism

The nondeterministic throughput of GRAND decoders paired with the regular data vector arrival rate I into the FIFO-Scheduling system, motivates the need for input buffering by means of a FIFO to avoid dataloss in the case of the decoders not terminating in time to accommodate the next datavector. As explained in section 2.3.3, if the ORBGRAND units keep iterating on the same data vectors over a significant time span, the regularly incoming and stowed $\mathbf{y}$ could fill all F slots of the FIFO, causing the system to become unaccommodating for incoming data vectors resulting in input data loss. To mitigate this unwanted effect, an early termination mechanism is employed by a control unit as described in [20], which in this work is implemented within the FIFO, which guarantees that incoming data vectors $\mathbf{y}$ can always be accommodated.

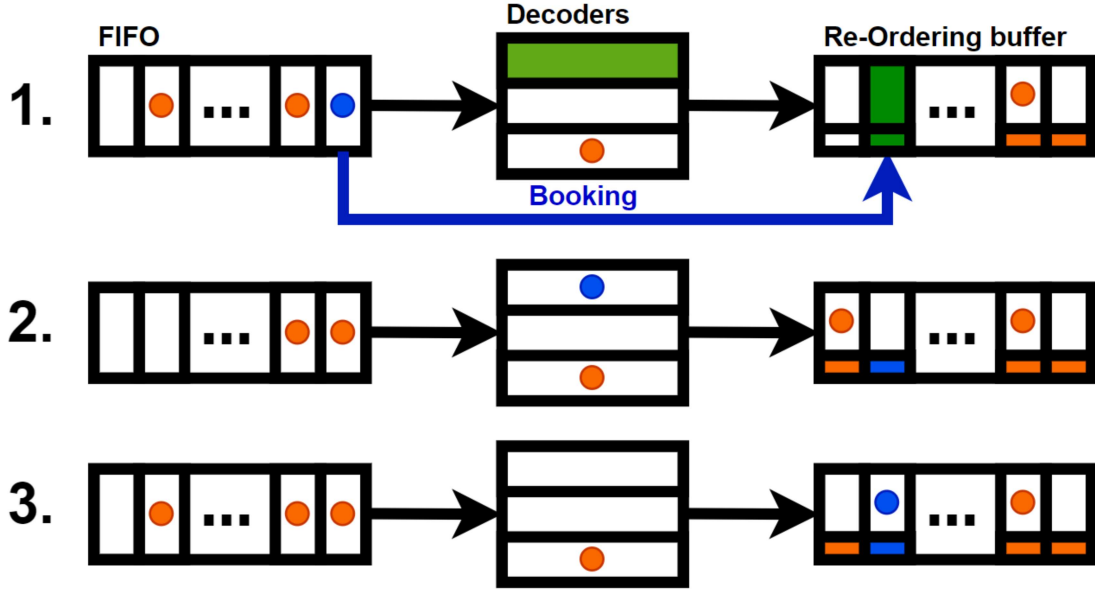


Figure 4.2: Booking mechanism.

In the implementation of the FIFO-Scheduling system given in this work, the FIFO is able to pop a data struct only when a decoder is free and a slot within the re-ordering buffer is available to be booked. Both of these conditions have to be fulfilled if a FIFO slot is to be popped, and both can cause the FIFO to fill up. A full FIFO is only problematic during the system cycle in which a data item arrives, and given that the input rate I is regular, the early termination mechanism is tuned to be triggered only when the FIFO is full upon pending data receipt, limiting the frequency of not strictly necessary early terminations. This is advantageous as each early terminated decoding unit is guaranteed to return an erroneous output, as the simulation will set the decoding correctness of the data struct to unsuccessful. In particular, the early termination mechanism as implemented in this work, is triggered if the FIFO is completely filled when a new data vector is expected within 2 system cycles, which allows the necessary measures to take place to free a slot in the FIFO just in time.

Once the early termination mechanism is triggered, the FIFO sets the longest running decoder to idle within the same cycle, which causes it to be freed by releasing the erroneous decoding in the re-ordering buffer through the distribution unit within the next cycle. As the longest running decoder is terminated, its decoding will be inserted in the front-most slot of the re-ordering buffer and thus released the cycle thereafter, allowing the FIFO to book a re-ordering buffer slot and be popped. This finally yields a free slot within the FIFO, allowing the system to accommodate the upcoming data vector. In total 2 cycles pass from the early termination triggering to the system being available to new data vectors. The entire early termination mechanism is listed below.

4 FIFO Scheduling Simulation

Early termination mechanism:

1. **cycle i**

- a) FIFO full and data item arrives in 2 cycles $\rightarrow$ Early termination triggered
- b) Longest running decoder set to idle

2. **cycle $i+1$**

- a) Early terminated decoding is also longest running decoding $\rightarrow$ unloaded into **first** re-ordering buffer slot

3. **cycle $i+2$**

- a) First Re-Ordering buffer slot full $\rightarrow$ re-ordering buffer is unloaded accommodating new booking
- b) FIFO is popped just in time for arriving data vector
- c) New data vector $\mathbf{y}$ pushed into FIFO

4.3 Performance indicators

To properly evaluate FIFO-Scheduling as a solution to the nondeterministic runtime of GRAND decodings, multiple parametrizations (I, F, D, R) are to be simulated and compared, which calls for relevant data to be collected during the simulations. As mentioned in section 4.1, the actual decoding of data vectors $\mathbf{y}$ is computed offline, allowing multiple FIFO-Scheduling architectures to be compared, while mitigating the need of repeating the computationally expensive ORBGRAND decoding each time.

Each offline decoded data vector is passed as a struct to the system with attributes indicating the number of required iterations until a valid decoding is found and the correctness of said decoding. The system then processes each data struct as described in sections 4.2.1 through 4.2.3, and modifies the decoding correctness attribute if an early termination happened, leading to the data struct being labeled as incorrectly decoded. For each data vector struct entering the system, the number of cycles passed within the system is measured, allowing for the latency of any given system parametrization to be assessed in detail. While a simulation of a given system parametrization is running, the occupancies of the FIFO, decoders and re-ordering buffer are collected over a predetermined number of system cycles.

Each offline decoded data vector is passed to the system as a struct, with attributes indicating the number of required iterations until a valid decoding is found and the correctness of the decoding. The system processes each data struct as described in sections 4.2.1 through 4.2.3, modifying the decoding correctness attribute if early termination

4 FIFO Scheduling Simulation

occurs, thereby labeling the data struct as incorrectly decoded. The total number of early terminations is registered.

For each data vector struct entering the system, the number of cycles within the system is measured, allowing for a detailed assessment of the latency for any given system parameterization. Furthermore, during the simulation of a given system parameterization, the occupancies of the FIFO, decoders, and reordering buffer are collected over a predetermined number of system cycles.

Once a representative number of data vectors has been processed by the system, the block error rate (BLER) is computed out of the decoding correctness attribute of all outputted data structs. Finally the number of early terminations is also gathered. These performance metrics provide insightful means of comparison amongst the simulated parametrizations as shown in section 5.2.

Results

5.1 ORBGRAND Results

The simulation implemented as detailed in section 3, is run on a Random Linear error correction Code (*RLC*) of $k = 234$ information bits and $n = 256$ encoding symbols. The resulting *BLER*, Eb/N_0 relationship as shown in figure 5.1a, and illustrates that a larger the number of reliability curve approximation segments, yields better decoding performance. Diminishing returns are achieved for *RLC*[234, 256] at $m \geq 4$ motivating the choice of $m = 4$ for the following analysis. The figure also clearly shows that the decoding performance improvement for larger m is particularly noticeable at higher Eb/N_0 . This result is due to the curviness of the reliability curve, which increases with Eb/N_0 , proving more demanding to accurately portray through a reduced number of segments, as was already shown in figures 2.1a and 2.1b. Given that ORBGRAND's noise effect sequence trials ordering relies on the approximation of the reliability curve, worse approximations result in worse decoding performance.

As shown in 2.2.2, larger reliability values signify more certainty in the detection accuracy of a particular data symbol. It follows that the approximation of the reliability curve at larger indexes, and thus larger reliability values, is less important than an accurate representation at the initial indexes. Using the definitions given in section 3.2, linear anchoring aims at approximating the whole of the reliability curve with similar accuracy, through regular sizing of the intervals. On the other hand, geometric anchoring approximates lesser reliabilities more accurately by doubling the interval size for each subsequent section starting from the lower indexes. This increased accuracy in the portrayal of the lower valued reliabilities, results in increased decoding performance, particularly noticeable at higher Eb/N_0 values. This result is illustrated in figures 5.1c and 5.1d respectively.

As mentioned in [9] and noticeable from figure 5.1d, a core feature of all GRAND algorithms is the rapid decline in the mean number of required iterations as Eb/N_0 increases.

5 Results

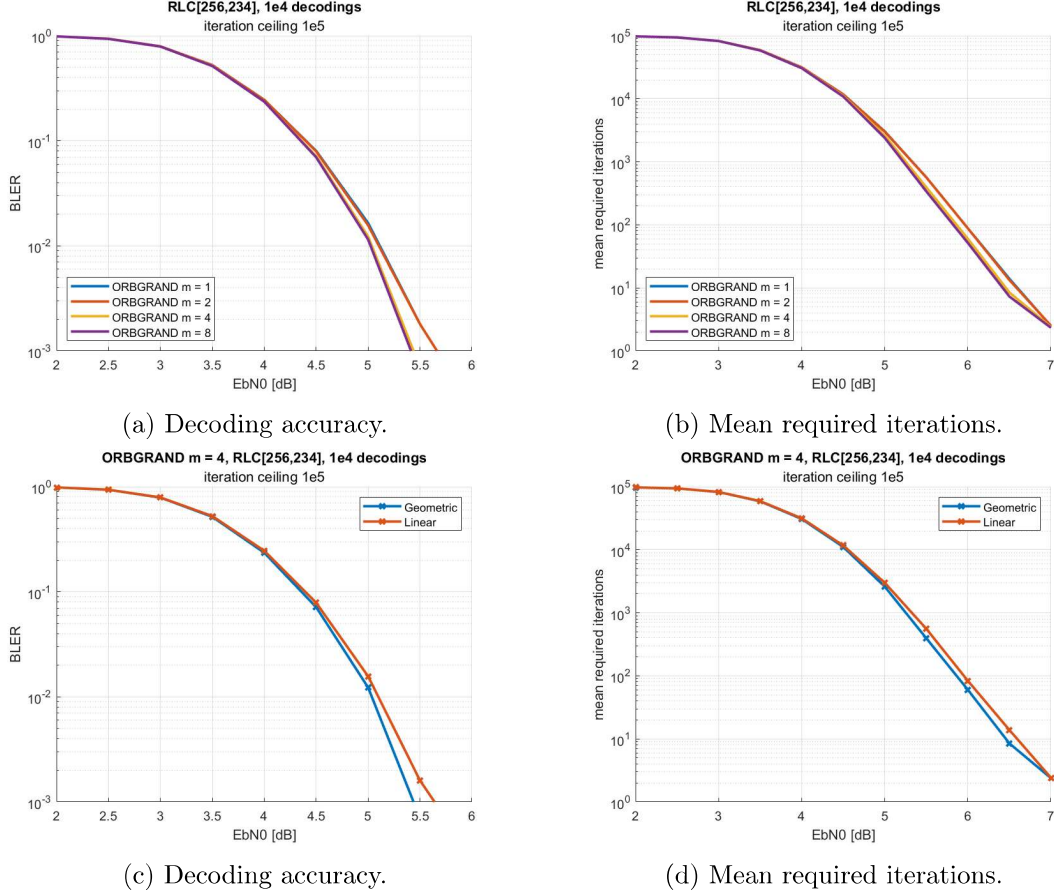


Figure 5.1: Comparison of decoding accuracy and mean required iterations across different segment numbers and anchoring methods.

Even more telling are the cumulative distribution function (CDF) curves of the required iterations at a given Eb/N_0 value, as provided by the figures 5.2a through 5.2d. These plots show that for all analysed noise levels Eb/N_0 , the majority of detected data vectors $\mathbf{y}$ are decoded within the first iteration, which corresponds to the case where $\mathbf{y} = \mathbf{x}$ and $\mathbf{z} = \{0\}^n$. As the number of required code-book checks increases, the associated data vectors diminish drastically, resulting in more than 90% of the decodings being found in less iterations than the mean μ_{trials} . This statistic indicates that few outliers require a disproportionate amount of noise effect sequence trials. As the Eb/N_0 diminishes, the CDF curve of the required iterations increases in steepness around the initial few iterations, drastically decreasing the mean.

Comparing the accuracy of decodings of different decoding complexities, as done in figure 5.3a, shows that as the iterations of a given decoding process increase, the probability of said decoding being successful decreases as already demonstrated in [9]. This further motivates the effectiveness of the early termination mechanism in the proposed FIFO-

5 Results

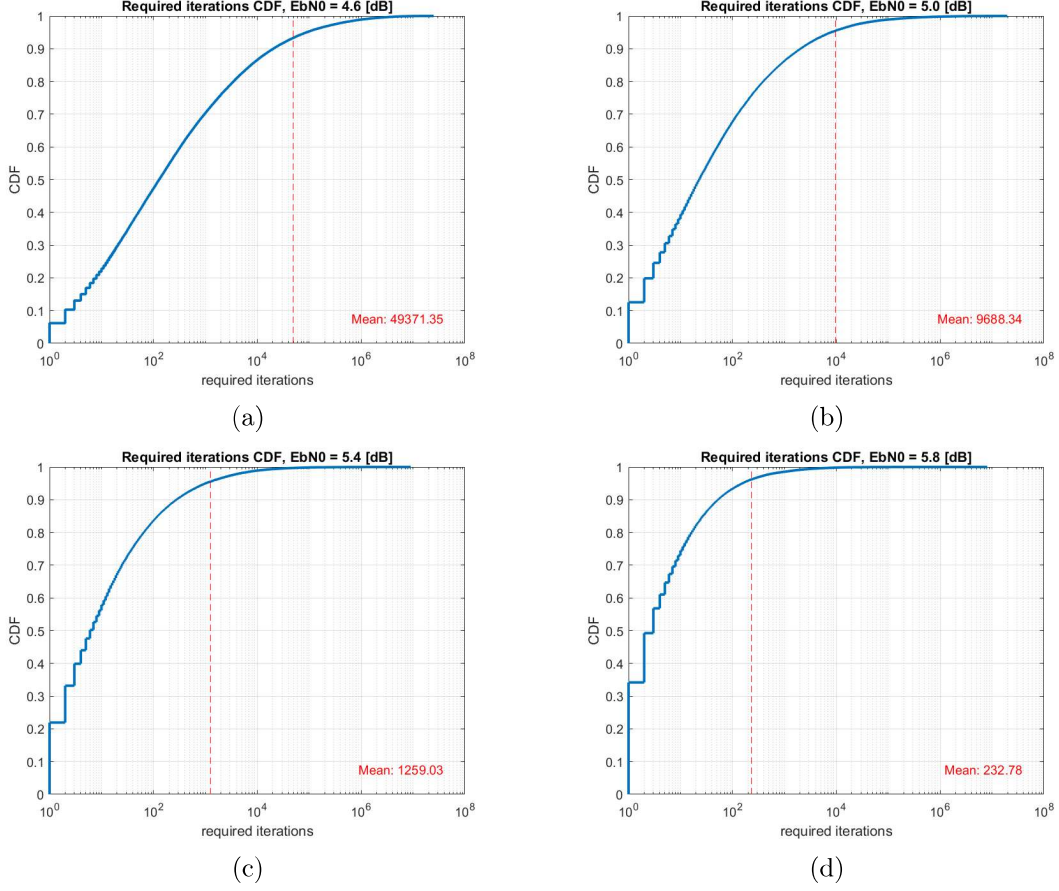


Figure 5.2: Required iterations CDF at different E_b/N_0 values. red dashed line at the mean. Decoder is geometrically anchored 4 segment ORBGRAND, for 10^4 decodings at iteration ceiling 10^9 .

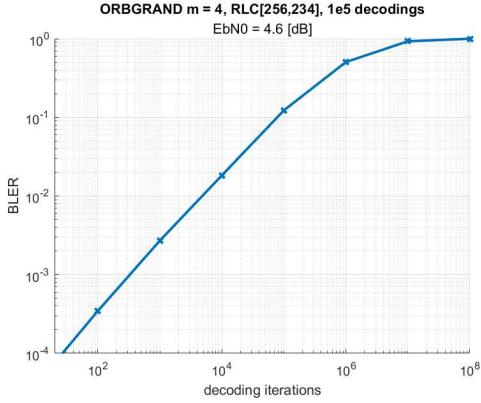
Scheduling architecture detailed in sections 2.3.3 and 4.2.5: terminating the longest running decoder leading to a decoding error is preferable to losing a new data vector at the input. This is because, as shown in (5.3a), the probability of decoding correctly the data vector within the longest iterating decoder, is on average less than the probability of decoding correctly a newly arrived data vector.

5.2 FIFO-Scheduling results

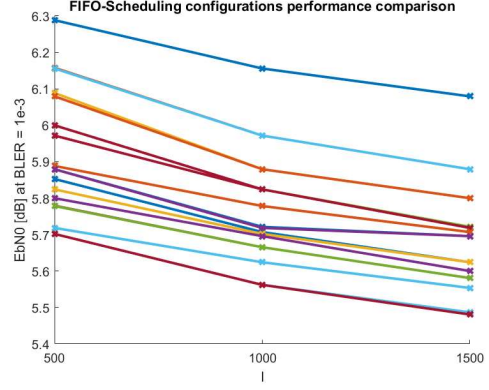
5.2.1 Overview

To properly evaluate FIFO-Scheduling as a viable approach to incorporate GRAND decoders in real-world systems reliant on deterministic throughput, several architecture

5 Results



(a) BLER at different numbers of required iterations. No decodings were abandoned.



(b) Relative performance of various parametrizations over different input rates.

parametrizations are simulated and compared. In this work, an ORBGRAND decoding process configured as detailed in section 5.1 is simulated for 10^5 data vectors, for $Eb/N0 \in \{4.6, 4.8, 5.0, \dots, 7.0\}$ and limited to 10^9 noise effect sequence trials. The resulting decodings are saved and 10^4 of them are fed as input to the different FIFO-Scheduling system configurations, as detailed in section 4.1.

From the resulting $BLER$ vs. $Eb/N0$ relationship, the $Eb/N0$ value where the simulated ORBGRAND decoder achieves $BLER = 10^{-3}$ is found by means of linear interpolation, resulting in $Eb/N0 \approx 5.1972$ [dB]. For this value, the mean required noise effect sequence trials for the given ORBGRAND configuration at 5.1972 [dB] is linearly interpolated, approximatively amounting to $\mu_{trials} \approx 3687$ [trials]. The FIFO-Scheduling architectures where then simulated for any valid parameter combination of ranges of input rate $I \in \{500, 1000, 1500\} \left[\frac{\text{cycles}}{\text{data vector}} \right]$, FIFO sizes $F \in \{0, 1, 2, 4, 8\}$ [slots], parallel decoders $D \in \{1, 2, 4, 8\}$ [decoders] and re-ordering buffer sizes $R \in \{0, 1, 2, 4, 8\}$ [slots]. The validity of each parametrization was determined according to section 4.2. From the resulting $BLER$ vs. $Eb/N0$ relationship of each valid parametrization, the linearly interpolated $Eb/N0$ value at which any given FIFO-Scheduling system reaches a $BLER = 10^{-3}$ is calculated. This $BLER$ intersect interpolation process is depicted in figures 5.4a through 5.4c, for the input rates $I \in \{500, 1000, 1500\} \left[\frac{\text{cycles}}{\text{data vector}} \right]$ expressed in $\left[\frac{\text{bit}}{\text{s}} \right]$, through a conversion explained in (5.2).

From the figures 5.4a-5.4c, it can be observed that over all three analysed throughput ratios, the performance of the reference is approximatively 0.3 – 1 [dB] better compared to all FIFO-Scheduling parametrizations. The accuracy degradation associated with the scheduling architecture is due to the rigorously imposed throughput, which triggers the early termination mechanism, tanking decoding performance. This observation also explains the decline in decoding accuracy associated with higher throughput ratios, as faster input rates cause the FIFO to back up more frequently, thereby triggering more

5 Results

early terminations. Furthermore, it can be observed that the best and worst performing simulated parametrizations, differ by approximatively 0.6 [dB]. Finally, in figure 5.3b it is shown that the relative performance of the parametrizations, does not differ over the various simulated input rates. For this reason, the following analysis is only shown for $I = 1000 \left[\frac{\text{cycles}}{\text{data vector}} \right]$.

To achieve a more insightful analysis, the need arises for the simulation parameters (I, F, D, R) to be converted into real-world values. Doing so requires a reference hardware implementation of ORBGRAND, such as the ones given in [19], out of which design $D1$ was chosen as it presents a comparable configuration of ORBGRAND as the one utilized in this work, yielding the following data:

$$\left\{ \begin{array}{l} \text{Decoder Operating frequency} = 746 \text{ [MHz]} \\ \text{Decoder Trials per clock cycle} = 4 \left[\frac{\text{trials}}{\text{clk cycle}} \right] \\ \text{Decoder size} = 0.97 \text{ [mm}^2\text{]} \\ \text{Bits per LLR} = 5 \text{ [bits]} \\ \text{TSMC-65nm D-FF area} = 8 \text{ [}\mu\text{m}^2\text{]} \end{array} \right. \quad (5.1)$$

Where for the number of noise effect sequence trials per clock cycle, the degree of parallelization P from design $D1$ was utilized as an approximation. Furthermore the Decoder Operating frequency was chosen to represent the operating frequency of the whole system, including the FIFO, distribution unit, collection unit and re-ordering buffer.

The mean number of required code-book membership tests executed for a given noise level $Eb/N0$ as shown in figures 5.2a through 5.2d, was found by letting each ORBGRAND decoding process iterate until a valid decoding was found. This performance could in principle be achieved through FIFO-Scheduling by assuming infinite FIFO slots feeding into a single decoder. Given that this situation is ideal, it proves useful to compare the simulated input rate against what is achievable in the ideal case, to assess how much of a system's theoretical decoding capacity is being used, assuming a single decoder. This motivates the following definitions (5.2), for the maximum possible mean throughput $\mu_{max_{TP}}$ of a single ORBGRAND unit, the actual input throughput $\mu_{in_{TP}}$ and their ratio $\mu_{ratio} = \frac{\mu_{in_{TP}}}{\mu_{max_{TP}}}$:

$$\begin{aligned} \mu_{max_{TP}} &= \frac{\text{Decoder Trials per clock cycle} \cdot \text{Decoder Operating Frequency}}{\mu_{trials}} \cdot k \text{ [Mbps]} \\ \mu_{in_{TP}} &= \frac{\text{Decoder Operating Frequency}}{I} \cdot k \text{ [Mbps]} \end{aligned} \quad (5.2)$$

For the chosen input rate $I = 1000 \left[\frac{\text{cycles}}{\text{data vector}} \right]$, and the data from (5.1), this amounts to $\mu_{max_{TP}} \approx 190 \text{ [Mbps]}$, $\mu_{in_{TP}} \approx 170 \text{ [Mbps]}$, resulting in a ratio of $\mu_{ratio} = 0.92$. This

5 Results

means that a single decoder configuration, provided $F \rightarrow \infty$, would be able to provide an average throughput of 190 [Mbps], without having to trigger any early termination.

5.2.2 Latency

As detailed in section 4.3, the latency measured by the data vectors is given in system cycles passed within the FIFO-Scheduling configuration. Maximum latency is evaluated by taking the maximum registered latency $L_{\max}$ for a given parametrization amongst all simulated data vectors, over all $Eb/N0$ values. This value is subsequently converted to the maximum latency of the system expressed in seconds $L_{\max}$ as follows:

$$L_{\max} = \frac{\text{Most cycles within the system}}{\text{Decoder operating frequency}} [\text{s}] \quad (5.3)$$

Setting the interpolated $Eb/N0$ value of the $BLER = 10^{-3}$ intersect of each parametrization (F, D, R) on the x-axis, and the associated maximal latency $L_{\max}$ on the y-axis, yields figure 5.5.

From plot 5.5, it can be observed that the best decoding performance is achieved by architecture configurations with the highest maximum latency, while the worse decoding performance is associated with the lowest maximum latency. Furthermore, all architectures seem to lie on the pareto optimal frontier, which implies that performance is traded for latency. Finally, it is noteworthy that the performance gains diminish with larger architectures, while the overhead of $L_{\max}$ increases significantly.

5.2.3 Area

The total area A_{tot} associated to any FIFO-Scheduling configuration (F, D, R) , is calculated as the sum of the partial areas of the FIFO A_{FIFO} , the area of the decoders $A_{decoder}$ and the area of the re-ordering buffer $A_{\text{re-ordering buffer}}$. Given that the input FIFO has to contain soft decoding information in order for ORBGRAND to work, it follows that each slot has to accomodate (Bits per LLR) [bits] for each of the n encoded symbols. The total area occupied by the decoders can be obtained by simple multiplication amongst its size and the decoder number D . Decoded data vectors only occupies k information bits, which implies that the R re-ordering buffer slots only have to accomodate for k bits each. Finally, the total area A_{tot} is given by the sum of these three components, as shown in equation (5.4).

5 Results

$$\begin{aligned}
A_{FIFO} &= F \cdot n \cdot (\text{Bits per LLR}) \cdot (\text{TSMC-65nm D-FF area}) \quad [\text{mm}^2] \\
A_{decoder} &= D \cdot (\text{Decoder size}) \quad [\text{mm}^2] \\
A_{\text{re-ordering buffer}} &= R \cdot k \cdot (\text{TSMC-65nm D-FF area}) \quad [\text{mm}^2] \\
A_{tot} &= A_{FIFO} + A_{decoder} + A_{\text{re-ordering buffer}} \quad [\text{mm}^2]
\end{aligned} \tag{5.4}$$

As done for the maximal latency, by setting the interpolated Eb/N_0 value of the $BLER = 10^{-3}$ intersect of each parametrization (F, D, R) on the x-axis, and the associated system area A_{tot} on the y-axis, results in the plot 5.6.

From equation (5.4), the impact on the total area of a single FIFO slot, re-ordering buffer slot and decoder can be computed as follows:

$$\begin{aligned}
A_{decoder} &= D \cdot (\text{Decoder size}) \quad [\text{mm}^2] = 0.97 \quad [\text{mm}^2] \\
A_{FIFO} &= F \cdot n \cdot (\text{Bits per LLR}) \cdot (\text{TSMC-65nm D-FF area}) \quad [\text{mm}^2] \\
&= 1 \cdot 256 \cdot 5 \cdot 8 \cdot 10^{-6} = 0.01 \quad [\text{mm}^2] \\
A_{\text{re-ordering buffer}} &= R \cdot k \cdot (\text{TSMC-65nm D-FF area}) \quad [\text{mm}^2] = 1 \cdot 234 \cdot 8 \cdot 10^{-6} \\
&= 0.0019 \quad [\text{mm}^2]
\end{aligned} \tag{5.5}$$

This indicates that the decoder area dominates the total system area, provided a modest number of FIFO slots F and re-ordering buffer slots R are utilized. This observation is clearly depicted in figure 5.6, and causes the decoding performance for any given approximate area value to vary considerably, leading to many architectures being far from the pareto frontier. Nevertheless, the worst and best performing parametrizations present the smallest and largest area respectively. Finally, the benefit of a larger number of buffering slots F and R , together with their trifling area overhead is clearly observable.

5.2.4 Storage

The storage S_{tot} required by each FIFO-Scheduling parametrization (F, D, R) is calculated as the sum of the extra storage at the input due to the FIFO S_{FIFO} , and the extra storage at the output due to the re-ordering buffer $S_{\text{re-ordering buffer}}$. As described in section 5.2.3, each FIFO slot has to accomodate n LLR values, which require (Bits per LLR) [bits] each. The re-ordering buffer, being situated after the decoding process, only has to store k information bits per slot, resulting in the equation below:

5 Results

$$\begin{aligned}
S_{FIFO} &= F \cdot n \cdot \text{Bits per LLR [bits]} \\
S_{\text{re-ordering buffer}} &= R \cdot k \text{ [bits]} \\
S_{tot} &= S_{FIFO} + S_{\text{re-ordering buffer}} \text{ [bits]}
\end{aligned} \tag{5.6}$$

As done for the maximal latency and the area, the storage required for each iteration can be represented meaningfully by setting the interpolated $Eb/N0$ value of the $BLER = 10^{-3}$ intersect of each on the x-Axis and the total storage S_{tot} on the y-Axis, resulting in plot 5.7.

From plot 5.7, a clear dominance of the FIFO slot number in terms of the extra storage statistic can be observed, while the re-ordering buffer size only has a negligible impact. This stems from the fact that each FIFO slot requires more than 5 times the memory of a re-ordering buffer slot, as is shown in equation (5.7). The best and worst performing architectures correspond to the ones which demand the most and least extra storage respectively. Furthermore, from architectures $(8, 2 - 8, 8)$, it is apparent that increasing the number of decoders $D > 2$ does not provide any performance advantage. Finally, it appears that the furthest architectures from the pareto optimal frontier, are the ones which only include a single decoder, and do thus not make use of any decoding parallelism, e.g. $(4, 1, 0)$.

$$\begin{aligned}
S_{FIFO} &= F \cdot n \cdot \text{Bits per LLR[bits]} = 1 \cdot 256 \cdot 5 = 1280[\text{bits}] \\
S_{\text{Re-Orderingbuffer}} &= R \cdot k[\text{bits}] = 1 \cdot 234 = 234[\text{bits}] \\
\text{Storage ratio:} &= \frac{S_{FIFO}}{S_{\text{Re-Orderingbuffer}}} = \frac{1280}{234} \approx 5.47
\end{aligned} \tag{5.7}$$

5.2.5 Area-Time product

From section 5.2.3 it is known that area an architecture's area is mostly dictated by its decoding parallelism D , while from section 5.2.2, it was suggested that latency is mainly influenced by the memory sizes F and R . Taking both of these observations together, provides the motivation for evaluating the area-time product AT , which is a measure often used to assess hardware efficiency [29]. To obtain this value, for any given parametrization (F, D, R) the maximum latency L_{max} and area A_{tot} found as detailed in sections 5.2.3 and 5.2.2, are combined in a single measure expressed in $[\text{mm}^2 \cdot s]$ and computed as follows:

$$AT = A_{tot} \cdot L_{max} [\text{mm}^2 \cdot s] \tag{5.8}$$

5 Results

Representing this value on a $BLER = 10^{-3}$ interest plot for each configuration as done for the maximum latency, area and storage in sections 5.2.2 through 5.2.4, yields plot 5.8.

The AT product plot 5.8, suggests that the best performing architecture $(F, D, R) = (8, 8, 8)$ is associated with the largest AT product, while the worst performing architecture $(F, D, R) = (0, 1, 0)$ corresponding to the hard-coded iteration ceiling approach, maps to the lowest AT product. Again, it can be observed that introducing a decoding parallelism $D > 2$, while associated with significant overhead, leads to negligible performance gains.

5.2.6 Early termination mechanism

Crucial to FIFO-Scheduling as introduced in [20] and detailed in sections 2.3.3, 4.2.5, is the early termination mechanism. This mechanism is triggered if the FIFO is full when a new data vector $\mathbf{y}$ is expected, which occurs when longer than usual decoding processes cause the system to back up. In figure 5.9, the occupied FIFO slots over a number of system cycles can be observed, where the red circles indicate the cycle where an early termination was triggered. Overall the early terminations and the times the FIFO is full, are registered over the course of a simulation run, which reveals that approximately 93.8% of the times the FIFO was full an early termination happened, corresponding to the times where a new data vector arrived.

Additionally, the speed at which the FIFO slots are freed is noteworthy. This is because most data vectors require only a few iterations to be decoded, as mentioned in section 5.1. Consequently, once a decoder begins processing stored data vectors, it is very likely to decode them quickly, thereby freeing up the FIFO slots.

5 Results

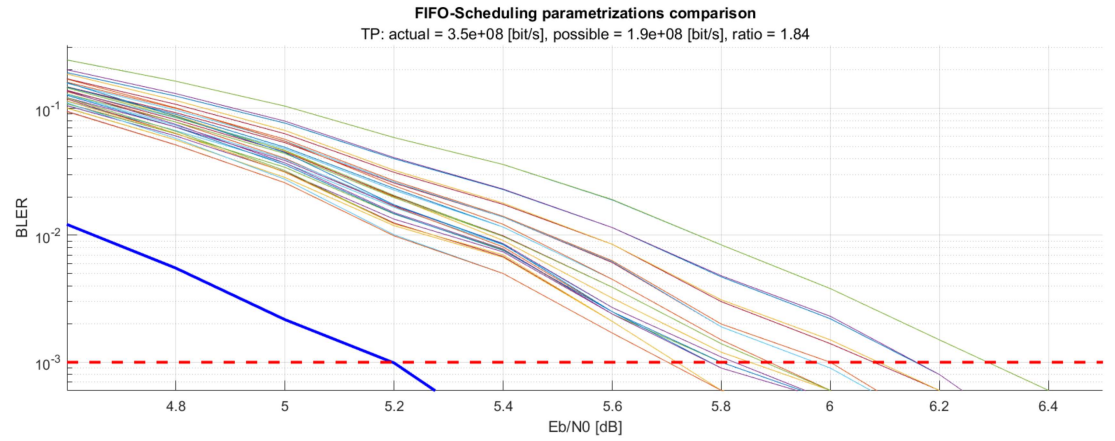
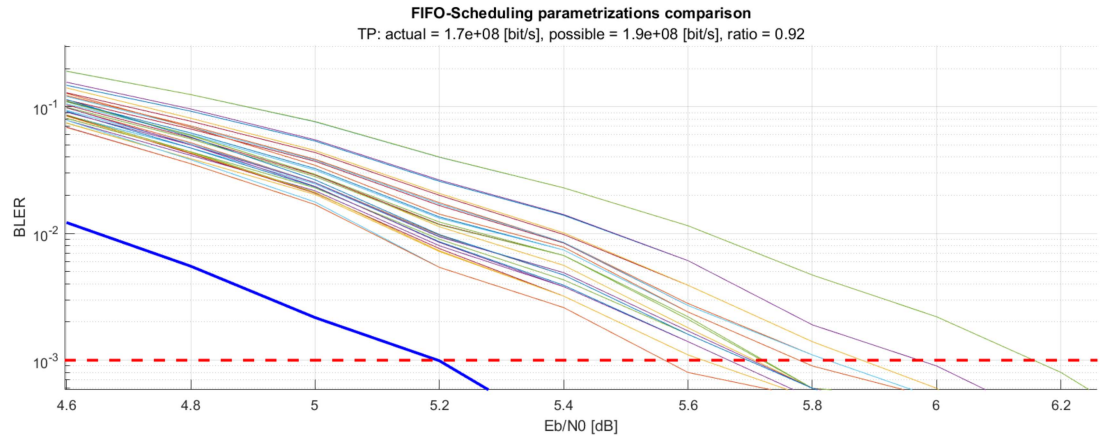
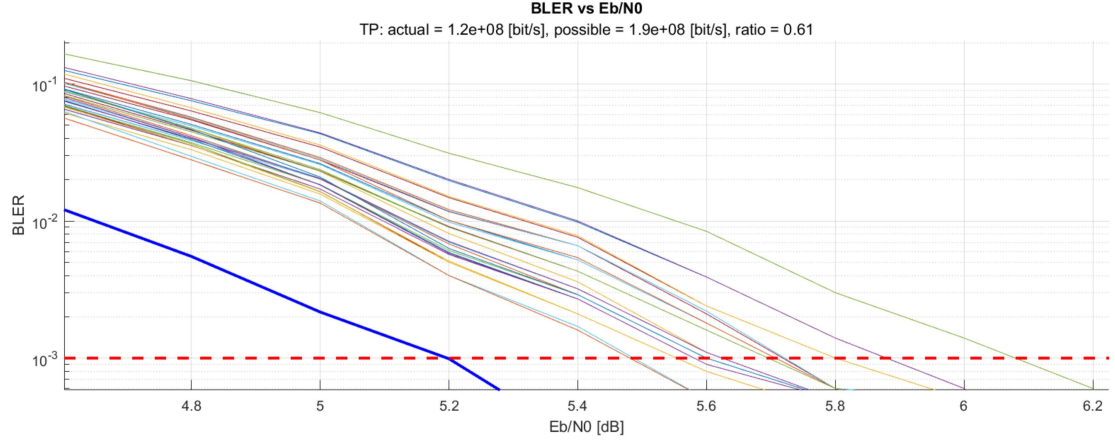


Figure 5.4: Contruction of $\text{BLER} = 10^{-3}$ intersect $\frac{Eb}{N0}$ data. Reference is given in blue.

5 Results

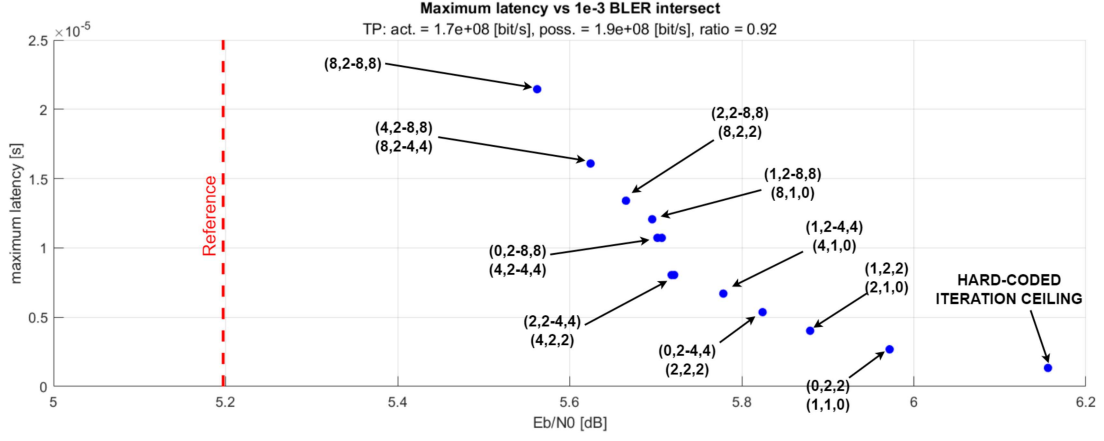


Figure 5.5: Pareto Maximum Latency plot, (F,D,R)

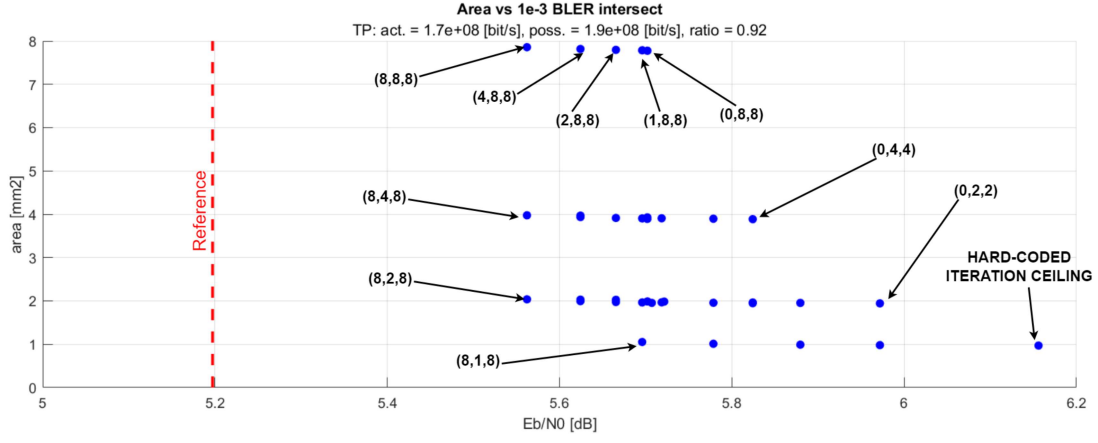


Figure 5.6: Pareto Area plot, (F,D,R)

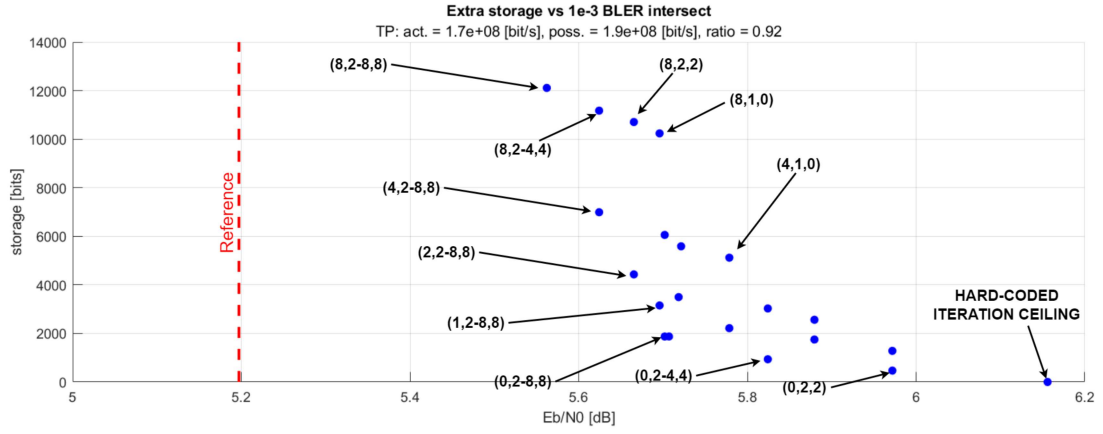


Figure 5.7: Pareto Storage plot, (F,D,R)

5 Results

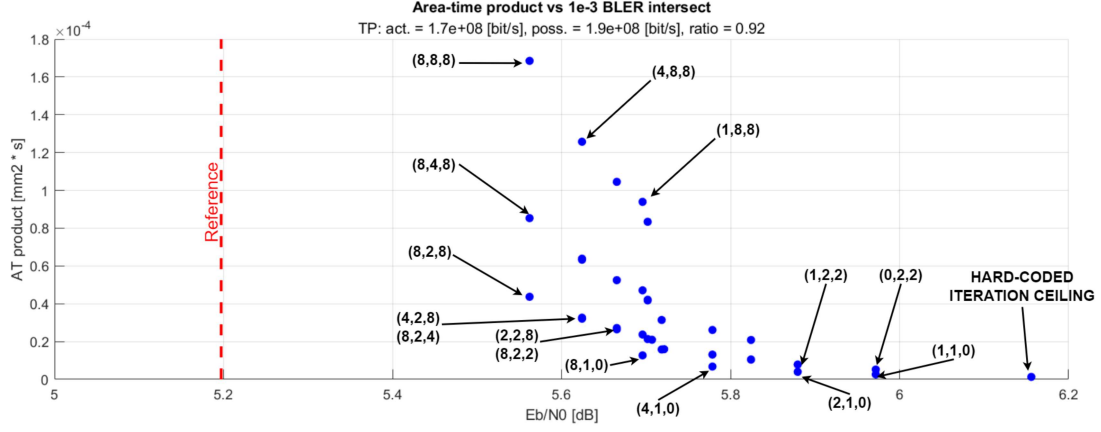


Figure 5.8: Pareto Area-Time product plot, (F,D,R)

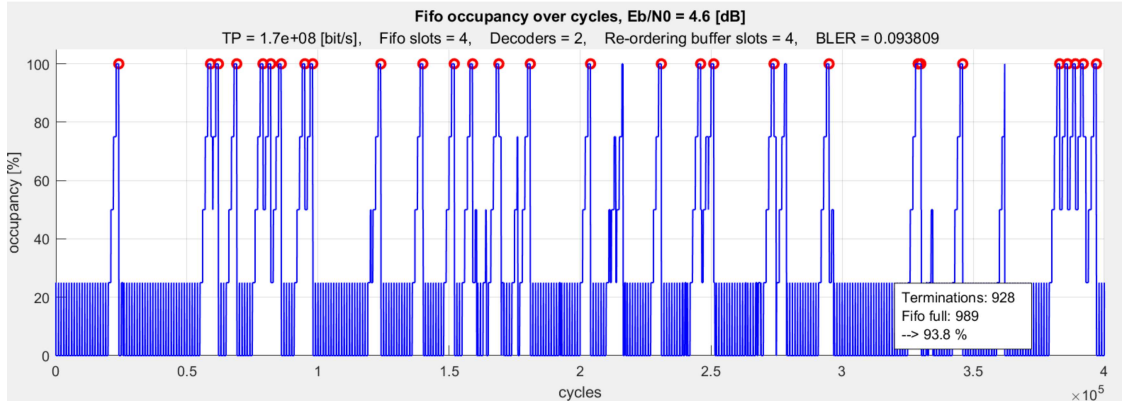


Figure 5.9: FIFO occupancy, over $4 \cdot 10^5$ cycles. Complete simulation involved 10^4 decodings.

Conclusion and Future Work

6.1 Summary

The aim of this work is to explore FIFO-Scheduling as an approach to mitigate the nondeterministic decoding latency inherent to GRAND algorithms. In particular, the soft decoding ORBGRAND is implemented and simulated, as described in section 3, confirming performance statistics and results first found in [9]. This implementation is then used to produce statistics on the ORBGRAND decoding process and to generate pre-decoded data as detailed in section 4.1, in order to efficiently simulate several FIFO-Scheduling architecture parametrizations.

The FIFO-Scheduling system, as first introduced in [20], is implemented as a combination of input buffering by means of a FIFO, parallel decoding and output data re-ordering through a re-ordering buffer. This re-ordering system is implemented as detailed in section 4.2.4 by means of a booking mechanism, which allows input data vectors $\mathbf{y}$ to be decoded in arbitrary order while being released from the system in the original ordering. Finally, the FIFO-Scheduling implementation includes an early termination mechanism first introduced in [20], as to avoid input data loss. Each valid parametrization of the FIFO-Scheduling system of the input rate I , FIFO slot number F , decoder number D and re-ordering buffer size R , is simulated for a range of Eb/N_0 values, and for each of which performance statistics are collected. The simulation parametrizations and the resulting performance statistics, are converted to real-world units by means of a reference hardware implementation of ORBGRAND [19], enabling comparisons amongst architectures to be made.

6.2 Conclusions

6.2.1 ORBGRAND

The ORBGRAND simulation as detailed in 3, is utilized to recreate results from [9].

From graph 5.1a, it can be concluded that for the chosen error correction code *RLC*[234, 256], a meaningful improvement in performance associated with an increased reliability curve approximation segment number m , is only obtained for $m \leq 4$, as previously shown in [9]. Additionally, it was shown that segmentations which focus on accurately approximating the lower end of the reliability curve, by means of geometric anchoring, result in superior decoding performance compared to regularly spaced intervals, such as linear anchoring, which is an effect especially noticeable at higher Eb/N_0 , confirming results from [9].

Furthermore, the distribution of the required number of code-book checks for ORBGRAND decodings at several Eb/N_0 values was explored, by generating CDF plots. It is shown that for the simulated range Eb/N_0 , more than 90% of the decodings occur in less than the mean required iterations, indicating that few outliers require a disproportionate amount of noise effect trials. Finally the number of required iterations for a decoding is demonstrated to be correlated to the accuracy of that decoding, which confirms statistics first found in [9].

6.2.2 FIFO-Scheduling

FIFO-Scheduling, as first introduced in [20], is shown to be an effective solution to harmonize intrinsically nondeterministic GRAND decoders with regular throughput demanding, real-world systems. Even the most limited FIFO-Scheduling architectures, employing a single FIFO slot and a single decoder, are shown to provide a performance advantage of approximately 0.2 [dB] over a single decoder, with a hard-coded iteration ceiling. Furthermore, it is shown that introducing parallelism by using more than one decoder, yields significant performance improvements, if the number of FIFO slots and re-ordering buffer slots is chosen to be appropriately large. The best performing simulated FIFO-Scheduling parametrizations improve performance up to 0.6 [dB], compared to a decoder with a hard-coded iteration ceiling. This performance improvement is limited to two decoders, as increasing the parallelism further is shown to have trifling performance benefits for the simulated parameter ranges, while significantly impacting overhead. This is particularly relevant when considering the area of the system, as comparable proposed VLSI implementations of ORBGRAND, such as design *D* from [19], dwarf the areas associated to the buffering slots.

6.3 Future Work

Utilizing FIFO-Scheduling as means of integrating nondeterministic iterative decoders such as GRAND in real-world systems should be explored further by means of simulation.

Firstly, more thorough statistical analysis of possible relationships between the shape of the reliability curve, the required code-book membership tests and the resulting decoding success should be performed as to explore the possibility of more effective early termination mechanisms to improve overall decoding performance. From this analysis, an input rejection mechanisms could arise, which would refuse access to the system to any data vector deemed likely to be decoded correctly by the system.

Secondly, more and larger FIFO-Scheduling parametrizations should be evaluated over a greater range of input rates, to obtain a more complete picture of the tradeoffs shown in this work. Finally, more variations to the architecture given in [20] could be explored, which could permit more complex scheduling mechanism, possibly bettering the decoding performance of the system.

6 *Conclusion and Future Work*

Bibliography

- [1] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [2] E. R. Berlekamp, R. J. McEliece, and H. C. A. van Tilborg, “On the inherent intractability of certain coding problems,” *IEEE Transactions on Information Theory*, vol. 24, no. 3, pp. 384–386, 1978.
- [3] S. Lin and D. J. Costello, *Error Control Coding: Fundamentals and Applications*. Englewood Cliffs, NJ, USA: Prentice Hall, 2004.
- [4] G. Durisi, T. Koch, and P. Popovski, “Toward massive, ultra-reliable, and low-latency wireless communication with short packets,” *Proceedings of the IEEE*, vol. 104, no. 9, pp. 1711–1726, Sep. 2016.
- [5] C. She, C. Yang, and T. Q. Quek, “Radio resource management for ultra-reliable and low-latency communications,” *IEEE Communications Magazine*, vol. 55, no. 6, pp. 72–78, Jun. 2017.
- [6] H. e. a. Chen, “Ultra-reliable low latency cellular networks: Use cases, challenges and approaches,” *IEEE Communications Magazine*, vol. 56, no. 12, pp. 119–125, Dec. 2018.
- [7] I. Parvez, A. Rahmati, I. Guvenc, A. I. Sarwat, and H. Dai, “A survey on low latency towards 5g: Ran, core network and caching solutions,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3098–3130, oct-dec 2018.
- [8] M. Médard, “Is 5 just what comes after 4?” *Nature Electronics*, vol. 3, no. 1, pp. 2–4, 2020.
- [9] K. Duffy, W. An, and M. Médard, “Ordered reliability bits guessing random additive noise decoding,” *IEEE Transactions on Signal Processing*, vol. 70, pp. 4528–4542, Feb. 2022.

Bibliography

- [10] K. R. Duffy, J. Li, and M. Médard, “Guessing noise, not code-words,” in *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, 2018, pp. 671–675.
- [11] K. R. Duffy, L. J., and M. Médard, “Capacity-achieving guessing random additive noise decoding,” *IEEE Transactions on Information Theory*, vol. 65, no. 7, pp. 4023–4040, Jul. 2019.
- [12] S. M. Abbas, T. Tonnellier, F. Ercan, and W. J. Gross, “High-throughput vlsi architecture for grand,” in *Proceedings of the IEEE Workshop on Signal Processing Systems (SiPS)*, 2020, pp. 1–6.
- [13] S. M. Abbas, M. Jalaleddine, and W. J. Gross, “High-throughput vlsi architecture for grand markov order,” in *Proceedings of the IEEE Workshop on Signal Processing Systems (SiPS)*, 2021, pp. 158–163.
- [14] A. e. a. Riaz, “Multi-code multi-rate universal maximum likelihood decoder using grand,” in *Proceedings of the IEEE 47th European Solid State Circuits Conference (ESSCIRC)*, 2021, pp. 239–246.
- [15] K. R. Duffy, “Ordered reliability bits guessing random additive noise decoding,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 8268–8272.
- [16] S. M. Abbas, T. Tonnellier, F. Ercan, M. Jalaleddine, and W. J. Gross, “High-throughput vlsi architecture for soft-decision decoding with orbgrand,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 8288–8292.
- [17] C. Condo, V. Bioglio, and I. Land, “High-performance low-complexity error pattern generation for orbgrand decoding,” in *Proceedings of the IEEE Global Communications Conference Workshops (GLOBECOM)*, 2021, pp. 1–16.
- [18] C. Condo, “A fixed latency orbgrand decoder architecture with lut aided error-pattern scheduling,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 5, pp. 2203–2211, May 2022.
- [19] C. Ji, X. You, C. Zhang, and C. Studer, “Efficient orbgrand implementation with parallel noise sequence generation,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, to appear, under submission.
- [20] C. Studer, “Iterative mimo decoding: Algorithms and vlsi implementation aspects,” Ph.D. dissertation, ETH Zurich, Zurich, CH, 2009.
- [21] W. An, M. Médard, and K. R. Duffy, “Keep the bursts and ditch the interleavers,” in *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, 2020, pp. 1–6.

Bibliography

- [22] ———, “Keep the bursts and ditch the interleavers,” *IEEE Transactions on Communications*, vol. 70, no. 6, pp. 3655–3667, Jun. 2022.
- [23] A. B. Cooper, “Soft decision decoding of block codes,” U.S. Army Laboratory Command, Ballistic Research Laboratory, Aberdeen Proving Ground, Maryland, Tech. Rep. BRL-MR-3721, 1988.
- [24] T. Kaneko, T. Nishijima, and S. Hirasawa, “An improvement of soft decision maximum-likelihood decoding algorithm using hard-decision bounded-distance decoding,” *IEEE Transactions on Information Theory*, vol. 43, no. 4, pp. 1314–1319, Apr. 1997.
- [25] V. Guruswami and M. Sudan, “Improved decoding of reed-solomon and algebraic-geometry codes,” *IEEE Transactions on Information Theory*, vol. 45, no. 6, pp. 1757–1767, Jun. 1999.
- [26] K. R. Duffy and M. Médard, “Guessing random additive noise decoding with soft detection symbol reliability information,” in *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, 2019, pp. 480–484.
- [27] K. R. Duffy, M. Médard, and W. An, “Guessing random additive noise decoding with symbol reliability information (srgrand),” *IEEE Transactions on Communications*, vol. 70, no. 1, pp. 3–18, Jan. 2022.
- [28] A. Solomon, K. R. Duffy, and M. Médard, “Soft maximum likelihood decoding using grand,” in *Proceedings of the IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.
- [29] H. Kaeslin, *Digital Integrated Circuit Design: From VLSI Architectures to CMOS Fabrication*. Cambridge University Press, Apr. 2008.